

Hamiltonian Hyperparameter Dynamics for Self-Tuning Hyperparameter Optimization of Neural Networks

Desai Kartik Amit¹, Bharat Soni^{1*}

¹Indian Institute of Information Technology Raichur, Raichur, Karnataka, India - 584135

Abstract

Hyperparameter optimisation (HPO) is a persistent bottleneck in modern machine learning: it governs both the convergence behaviour and the final accuracy of a trained model, yet it is almost always treated as an outer loop that is mechanically decoupled from the training dynamics it controls. Grid and random search scale poorly with dimension; Bayesian optimisation (BO) improves sample efficiency through a Gaussian-process (GP) surrogate but still requires a fresh training run, and a cubic-in-trials GP refit, for every candidate configuration it proposes. Adaptive first-order optimisers such as Adam and quasi-Newton methods such as L-BFGS accelerate the inner training loop itself but say nothing about how hyperparameters ought to evolve while that loop runs. Symplectic and variational integrators have separately been shown to turn optimisation into a structure-preserving dynamical system—most notably through the adaptive Hamiltonian variational integrators of Duruisseaux, Schmitt, and Leok [18], which integrate the *weights* of a model along a Bregman Hamiltonian flow—but that line of work treats hyperparameters as fixed externalities of the dynamical system rather than as objects the dynamics could also govern. We close this gap by promoting hyperparameters to dynamical variables in their own right: **Hamiltonian Hyperparameter Dynamics (HHD)** couples weights θ and hyperparameters λ in a single augmented phase space and evolves both jointly under Hamilton’s equations, integrated symplectically by a leapfrog scheme. We benchmark this idea against **(B) a Hybrid Adam + L-BFGS + Bayesian-Optimisation (ABBO)** baseline that represents the strongest practical alternative, and against **(C) a Unified HHD-ABBO framework** that folds stochastic gradient descent, symplectic sampling, and curvature-based refinement into a single three-phase curriculum executed at every epoch, thereby eliminating BO’s outer retraining loop altogether. We evaluate all three approaches on a controlled physics benchmark (the harmonic-oscillator energy landscape), on a small, CPU-trainable slice of CIFAR-10, and on eleven standardised tabular benchmarks drawn from HPOBench, HPOLib, and NAS-Bench-201, the latter averaged over five random seeds with Friedman/Nemenyi significance testing. The Unified framework (Method C) attains the strongest reconstruction accuracy on the physics benchmark averaged over five seeds ($\text{MSE} = 0.0033 \pm 0.0001$, $R^2 = 0.99994 \pm 0.00001$), representing approximately a $14\times$ reduction in error relative to ABBO at a comparable wall-clock cost, while retaining symplectic energy conservation, smooth hyperparameter trajectories, and second-order curvature exploitation that pure HHD lacks. Across the eleven HPO benchmarks, Method B achieves the second-best average rank (2.82) and Method C the third (3.09), both ahead of Optuna’s Tree-structured Parzen Estimator (1.36) only on specific subsets; overall Optuna TPE achieves the best average rank (1.36), followed by Method B (2.82), Method C (3.09), random search (3.64), and Method A (4.09). The Friedman test across 11 datasets is significant ($p = 7.9 \times 10^{-4}$), and the Nemenyi CD at $\alpha = 0.05$ is 1.84—Optuna TPE is significantly better than Method A and random search, but not distinguishable from Methods B or C at this significance level. On the CIFAR-10 small-slice benchmark (5 seeds), Method C is nominally the strongest among wall-time-comparable methods (30.60%), though its interval overlaps Method A (30.90%); the two are treated as effectively tied on this small validation set. Taken together, these results extend the variational-integrator perspective on optimisation from the weight space alone to the joint weight-hyperparameter phase

*Corresponding author e-mail id: bharat25soni@gmail.com, bsoni@iiitr.ac.in

space, establishing a structured, interpretable, and computationally economical alternative to the conventional split between training and tuning.

1 Nomenclature

Table 1: **Complete Symbol Reference**

<i>Hamiltonian mechanics and phase space</i>			
q	Generalised position (phase-space coordinate)	p	Generalised momentum
θ	Network weight vector $\in \mathbb{R}^d$	λ	Hyperparameter vector $\in \mathbb{R}^k$
p_θ	Conjugate momentum for network weights	p_λ	Conjugate momentum for hyperparameters
m_θ	Effective mass (inertia) of weight momenta	m_λ	Effective mass (inertia) of HP momenta
H	Total Hamiltonian $= T + V$	$\tilde{H}$	Shadow Hamiltonian
T	Kinetic energy $= \sum p^2/(2m)$	V	Potential energy, $V = \mathcal{L}$ (training loss)
ε	Leapfrog step size	L	Leapfrog sub-steps per HMC proposal
ΔH	Hamiltonian error of a leapfrog proposal	$\mathcal{P}_P$	L-BFGS plateau buffer (length P)
<i>Optimisers: Adam and L-BFGS</i>			
η, α	Learning rates (Adam / L-BFGS line search)	$\mathcal{L}$	Mean-squared-error (MSE) loss
β_1, β_2	Adam moment decay rates	H_t^{-1}	L-BFGS inverse-Hessian approx.
m_t, v_t	Adam running moment estimates	P	Plateau patience window (epochs)
$\hat{m}_t, \hat{v}_t$	Bias-corrected Adam moments	τ	Plateau relative-loss tolerance
<i>Bayesian optimisation and Gaussian processes</i>			
$\kappa(\cdot, \cdot)$	GP kernel function (RBF)	$\mu(\lambda)$	GP posterior mean at λ
$\sigma(\lambda)$	GP posterior std. deviation	$\text{EI}(\lambda)$	Expected Improvement acquisition
Φ, ϕ	Standard normal CDF and PDF	f^*	Best validation loss so far
ξ	Exploration–exploitation bonus	σ_f^2	GP signal variance
ℓ	GP length-scale parameter	σ_n^2	GP observation noise variance
<i>Dimensions and evaluation metrics</i>			
N	Number of training samples	d	Dimensionality of weight space
k	Dimensionality of HP space	R^2	Coefficient of determination
MAE	Mean absolute error (energy landscape)	4 RMSE	Root-mean-squared error (energy landscape)

2 Introduction

2.1 The Genesis and Evolution of Hyperparameter Optimisation

Hyperparameter optimisation (HPO) emerged from the routine practice of tuning regularisation coefficients and similar quantities by hand or by exhaustive grid search [57, 58]. The field acquired its modern shape with Bergstra and Bengio [28], who showed that random search outperforms grid search by exploiting a property now taken for granted in HPO: most hyperparameter landscapes have low *effective* dimensionality, so unstructured sampling covers the influential directions more efficiently than a grid that wastes evaluations on directions with little effect on generalisation.

As architectures grew, the black-box nature of neural-network training rewarded methods that were both sample-efficient and able to incorporate information from earlier evaluations—conditions met by sequential model-based optimisation and, in particular, Bayesian optimisation (BO) [7]. A Gaussian-process (GP) surrogate over the objective, combined with an acquisition function such as Expected Improvement (EI) [6], steers the search toward promising regions while balancing exploration against exploitation. The price of this sample efficiency is paid in the outer loop: every trial demands a full re-initialisation and retraining of the underlying model, and the GP itself scales cubically in the number of observations already collected.

2.2 The Hamiltonian Tradition: From Mechanics to Optimisation

Hamiltonian mechanics traces back to Hamilton’s 1833 reformulation of Newtonian dynamics [56], which replaces forces and accelerations with energy conservation expressed in canonical coordinates (q, p) . Duane *et al.* [11] carried this structure into computational statistics with Hybrid Monte Carlo (now universally called Hamiltonian Monte Carlo, HMC), exploiting the symplectic geometry of the leapfrog integrator to let a Markov chain propose large, informed moves through a high-dimensional distribution without the diffusive random walk that limits Metropolis–Hastings. Neal [12] established HMC as the reference method for Bayesian posterior sampling, and Betancourt [13] clarified *why* symplecticity delivers this efficiency: the leapfrog map exactly conserves a *shadow Hamiltonian* close to the true one, which bounds the numerical energy error globally and keeps Metropolis acceptance rates high regardless of step size.

It is this symplectic machinery, rather than HMC’s sampling objective, that motivates the present work. Duruisseaux, Schmitt, and Leok [18] showed that Hamiltonian variational integrators—built from Type-II or Type-III generating functions rather than the traditional Lagrangian (Type-I) construction—can be made time-adaptive even though the Poincaré transformation that introduces a variable time-step renders the underlying Hamiltonian degenerate. Applied to the Bregman Hamiltonian of Wibisono, Wilson, and Jordan’s variational framework for accelerated optimisation [23], their adaptive integrators reproduce Nesterov-type acceleration as a symplectic flow in weight space and demonstrate, empirically, that a carefully tuned adaptive time-step outperforms a fixed one by a wide margin. Their Riemannian extension [19] carries the same idea onto curved manifolds, and França *et al.* [22] proved that the dissipative Hamiltonian systems natural to gradient descent still preserve a shadow Hamiltonian and retain their continuous-time convergence rate under symplectic discretisation—together supplying the theoretical guarantees on which the present framework draws.

2.3 From Weight-Space Dynamics to Joint Weight–Hyperparameter Dynamics

What none of this prior work disturbs is the boundary between *training* and *tuning*. The integrators of Duruisseaux *et al.* accelerate the descent of θ for a *fixed* hyperparameter vector λ ; the hyperparameters themselves remain outside the phase space, exactly as they do in Bayesian optimisation’s outer loop. Gradient-based HPO via implicit differentiation [24, 25, 26] narrows this gap by differentiating

through the fixed point $\nabla_{\theta}\mathcal{L}(\theta^*(\lambda), \lambda) = 0$, but it assumes exact convergence of the inner loop at every outer step and offers no geometric or probabilistic guarantee on the trajectory $\lambda(t)$ itself.

We ask a different question: *if Hamiltonian variational integration is the right tool for accelerating θ , why not let λ ride the same symplectic flow?* Treating the training loss as a potential energy surface $V = \mathcal{L}(\theta, \lambda)$ over the *joint* space of weights and hyperparameters, and introducing conjugate momenta $(p_{\theta}, p_{\lambda})$ for both, extends the adaptive-integrator programme of [18] along an axis it does not address: hyperparameters become particles co-evolving with the weights they govern, inheriting symplectic conservation and detailed balance directly from the leapfrog construction rather than from an outer BO loop. This is the construction we call **Hamiltonian Hyperparameter Dynamics (HHD)**. Because pure HHD, like pure HMC, can be slow to refine a solution once it is near an optimum, we further introduce a **Unified HHD-ABBO** framework that interleaves the leapfrog co-evolution with Adam micro-steps and plateau-triggered L-BFGS refinement inside a single epoch, retaining the structural guarantees of HHD while recovering the curvature-aware convergence speed of a quasi-Newton method—without ever leaving the training loop to retrain from scratch.

We test this idea on the harmonic-oscillator energy landscape used throughout the Hamiltonian-neural-network literature for its exact, analytically known ground truth [42, 44], on CIFAR-10 image classification, and on eleven standardised HPO benchmarks, comparing pure HHD (Method A), a Hybrid Adam + L-BFGS + BO baseline (Method B) that represents current best practice, and the Unified framework (Method C).

2.4 Contributions

This paper makes the following contributions:

- C1.** We formulate **Hamiltonian Hyperparameter Dynamics (HHD)** (Method A), which extends the adaptive Hamiltonian variational-integrator framework of [18] from weight-space dynamics alone to a joint phase space in which network weights *and* hyperparameters evolve together under symplectic leapfrog integration.
- C2.** We construct a **Hybrid ABBO baseline** (Method B) combining Adam, L-BFGS, and Bayesian optimisation, chosen as the strongest practically competitive reference against which to evaluate Method A and Method C.
- C3.** We introduce the **Unified HHD-ABBO framework** (Method C), which integrates all three optimisers into a single three-phase curriculum applied within every training epoch, inheriting symplectic conservation from Method A and second-order curvature exploitation from Method B while eliminating Method B’s expensive outer BO loop entirely.
- C4.** We provide **formal theoretical guarantees**—the shadow Hamiltonian error bound, the detailed balance of Method C’s Markov chain, and a per-epoch computational-cost comparison across all three methods—that mirror, in the joint weight–hyperparameter setting, the guarantees [18] established for weight-space adaptive integration alone.
- C5.** We provide a **multi-benchmark evaluation** spanning the harmonic-oscillator energy landscape (MAE, RMSE, R^2 , Hamiltonian conservation, hyperparameter-trajectory smoothness, wall time), CIFAR-10 classification, and eleven standardised tabular benchmarks from HPOBench, HPOLib, and NAS-Bench-201 [37, 39], with Friedman and Nemenyi statistical tests over five independent seeds.

2.5 Research Questions

- RQ1.** Can network weights and hyperparameters be jointly optimised via Hamiltonian dynamics, and does the resulting flow preserve physical properties such as energy conservation, as it does for weight-space integrators alone [18]?
- RQ2.** How does pure HHD compare, quantitatively and qualitatively, to the practically competitive ABBO baseline, and at what computational cost?
- RQ3.** Does the Unified method, by combining all three optimisers in a structured curriculum, outperform both individual frameworks across diverse benchmark categories?
- RQ4.** What unique theoretical or practical properties does each framework provide, and in which deployment scenarios should each be preferred?

3 Background and Theoretical Foundations

3.1 Adam Optimiser

Adam [1] maintains per-parameter adaptive learning rates through exponential moving averages of the gradient g_t and its element-wise square. Given gradient g_t at step t , the update rules are

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (1)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (2)$$

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \quad (3)$$

where the bias-corrected estimates are $\hat{m}_t = m_t / (1 - \beta_1^t)$ and $\hat{v}_t = v_t / (1 - \beta_2^t)$. We use the standard defaults $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\epsilon = 10^{-8}$ throughout. Adam is effective early in training, while the loss landscape remains far from any stationary point, and its $O(d)$ per-step cost suits mini-batch stochastic settings; it does not, however, exploit second-order curvature, which limits its rate of convergence near an optimum.

3.2 L-BFGS (Limited-Memory Broyden–Fletcher–Goldfarb–Shanno)

L-BFGS [2, 4] approximates Newton’s method for large-scale problems by implicitly representing the inverse Hessian H_t^{-1} through a rolling window of the m most recent gradient-difference pairs $\{(s_i, y_i)\}$, where $s_i = \theta_i - \theta_{i-1}$ and $y_i = \nabla \mathcal{L}(\theta_i) - \nabla \mathcal{L}(\theta_{i-1})$. The parameter update is

$$\theta_{t+1} = \theta_t - \alpha_t H_t^{-1} g_t, \quad (4)$$

with step size α_t chosen by a backtracking line search satisfying the strong Wolfe conditions [3]. Memory and compute scale as $O(md)$; we use history size $m = 10$ throughout. Near a stationary point, L-BFGS attains super-linear convergence but requires full-batch gradients.

3.3 Bayesian Optimisation with Gaussian Process Surrogates

Bayesian optimisation [7] models the black-box objective $f : \lambda \mapsto \mathcal{L}_{val}$ with a Gaussian-process prior [8]. We use the RBF (squared-exponential) kernel,

$$\kappa(\lambda, \lambda') = \sigma_f^2 \exp\left(-\frac{\|\lambda - \lambda'\|^2}{2\ell^2}\right) + \sigma_n^2 \delta_{\lambda\lambda'}. \quad (5)$$

After t observations $\mathcal{D}_t = \{(\lambda_i, f_i)\}_{i=1}^t$, the GP yields a closed-form posterior predictive distribution, and new candidates maximise the Expected Improvement acquisition [6],

$$\begin{aligned} \text{EI}(\lambda) &= (f^* - \mu(\lambda) - \xi)\Phi(Z) + \sigma(\lambda)\phi(Z), \\ Z &= \frac{f^* - \mu(\lambda) - \xi}{\sigma(\lambda)}, \end{aligned} \tag{6}$$

where f^* is the best validation loss observed so far, $\xi \geq 0$ trades exploration against exploitation, and Φ, ϕ are the standard normal CDF and PDF.

3.4 Hamiltonian Monte Carlo

HMC [11, 12] augments the target parameter space with auxiliary momenta $p \sim \mathcal{N}(0, M)$ and defines the extended energy

$$H(\theta, p) = \mathcal{L}(\theta) + \frac{1}{2}p^T M^{-1}p, \tag{7}$$

evolved under Hamilton’s equations and numerically integrated by the *leapfrog* (Störmer–Verlet) scheme [16]:

$$p_{t+\varepsilon/2} = p_t - \frac{\varepsilon}{2}\nabla_{\theta}\mathcal{L}(\theta_t) \tag{8}$$

$$\theta_{t+\varepsilon} = \theta_t + \varepsilon M^{-1}p_{t+\varepsilon/2} \tag{9}$$

$$p_{t+\varepsilon} = p_{t+\varepsilon/2} - \frac{\varepsilon}{2}\nabla_{\theta}\mathcal{L}(\theta_{t+\varepsilon}). \tag{10}$$

After L leapfrog steps, the proposed state (θ', p') is accepted with probability $\min(1, e^{-\Delta H})$. Because the leapfrog map is time-reversible and volume-preserving (symplectic), it conserves a *shadow Hamiltonian* $\tilde{H} = H + O(\varepsilon^2)$, formalised below following the backward error analysis underlying both HMC [12] and the adaptive integrators of [18].

Proposition 3.1 (Shadow Hamiltonian Bound [17, Thm. IX.3.1]). *Let $H : \mathbb{R}^{2n} \rightarrow \mathbb{R}$ be smooth with bounded derivatives. Under the leapfrog integrator with step size ε , there exists a smooth shadow Hamiltonian $\tilde{H}$ satisfying*

$$|\tilde{H}(\theta, p) - H(\theta, p)| = O(\varepsilon^2),$$

which is exactly conserved by the leapfrog map. Consequently the Hamiltonian error satisfies $|\Delta H| = O(\varepsilon^2)$ globally, and the Metropolis–Hastings acceptance rate stays bounded away from zero for every finite ε .

Remark 3.2 (Dissipative Extension). When Adam micro-steps (Phase 1 of Method C) are interleaved with leapfrog proposals, the joint system becomes a *dissipative* Hamiltonian system. By the theory of França *et al.* [22], symplectic discretisation of such systems preserves a modified shadow Hamiltonian $\tilde{H}_{\text{diss}}$ and maintains the continuous-time convergence rate up to a controlled error—justifying the claim that Method C retains shadow-Hamiltonian conservation despite the dissipation that Adam introduces.

4 Methodology

This section sets out the mathematical formulation that distinguishes the three frameworks. Concrete hyperparameter values and training settings are deferred to Section 7.

4.1 Method A: Hamiltonian Hyperparameter Dynamics (HHD)

4.1.1 Augmented Phase Space

The central move of Method A is to treat both network weights $\theta \in \mathbb{R}^d$ and hyperparameters $\lambda \in \mathbb{R}^k$ as *generalised positions* in a single joint phase space, each equipped with its own conjugate momentum. This is the same Hamiltonian construction that Duruisseaux *et al.* [18] apply to θ alone; here we extend the phase space to include λ as well, drawing on the physics-informed learning framework of Greydanus *et al.* [42] and Zhang *et al.* [44] for the underlying weight dynamics. The augmented Hamiltonian is

$$H(\theta, \lambda, p_\theta, p_\lambda) = \underbrace{\sum_i \frac{p_{\theta,i}^2}{2m_\theta} + \sum_j \frac{p_{\lambda,j}^2}{2m_\lambda}}_{T \text{ (kinetic)}} + \underbrace{\mathcal{L}(\theta, \lambda)}_{V \text{ (potential)}} , \quad (11)$$

with separate effective masses m_θ for weight momenta and m_λ for hyperparameter momenta. The mass ratio m_λ/m_θ sets the relative *timescale* of hyperparameter evolution: a smaller m_λ lets hyperparameters move faster than weights, exploring the hyperparameter landscape without destabilising weight convergence—an extra degree of freedom that has no counterpart in the single-particle weight dynamics of [18].

4.1.2 Hamilton’s Equations for the Joint System

Differentiating Eq. (11) with respect to the four canonical variables gives four coupled ordinary differential equations:

$$\dot{\theta} = + \frac{\partial H}{\partial p_\theta} = \frac{p_\theta}{m_\theta} \quad (12)$$

$$\dot{p}_\theta = - \frac{\partial H}{\partial \theta} = -\nabla_\theta \mathcal{L}(\theta, \lambda) \quad (13)$$

$$\dot{\lambda} = + \frac{\partial H}{\partial p_\lambda} = \frac{p_\lambda}{m_\lambda} \quad (14)$$

$$\dot{p}_\lambda = - \frac{\partial H}{\partial \lambda} = -\nabla_\lambda \mathcal{L}(\theta, \lambda). \quad (15)$$

Weight gradients $\nabla_\theta \mathcal{L}$ are computed by automatic differentiation; hyperparameter gradients $\nabla_\lambda \mathcal{L}$ are approximated by central finite differences [49], with perturbation δ_λ documented in Table 4.

4.1.3 Continuous Relaxation of Discrete Hyperparameters

Remark 4.1 (Finite-Difference Gradient Approximation). For a discrete hyperparameter λ_j with N_j valid values, the one-sided finite-difference approximation introduces a truncation error of $O(\delta)$ in the hyperparameter gradient, where $\delta = 1.0$ index step. The approximation is consistent in the limit as the configuration space becomes dense over the integer lattice.

Table 2: **Continuous Parameterisations for Discrete Hyperparameters**

Hyperparameter	Variable	Parameterisation	Range
Learning rate	λ_{lr}	$\log_{10}(\eta)$	$(-4, -1)$
Dropout rate	λ_{do}	p_{drop}	$(0, 0.3)$
Network depth	λ_{nl}	Continuous integer	$(1, 8)$
Layer width	λ_{nn}	Continuous integer	$(16, 256)$
Batch size	λ_{bs}	$\log_2(b)$	$(4, 6)$

After each accepted HMC proposal, continuous values are snapped to the nearest valid integer for architecture parameters before instantiating the network. For tabular benchmarks the index relaxation is

$$\lambda_j^{\text{discrete}} = \text{choices}_j[\text{round}(\text{clip}(\lambda_j, 0, N_j - 1))], \quad (16)$$

and hyperparameter gradients on lookup-table benchmarks use the one-sided finite difference

$$\frac{\partial \mathcal{L}}{\partial \lambda_d} \approx \frac{\mathcal{L}(\lambda + \delta e_d) - \mathcal{L}(\lambda)}{\delta}. \quad (17)$$

4.1.4 Two-Phase Training Procedure

Phase 1 (Adam warm-up). Adam trains the weights for N_{wu} epochs at fixed λ_0 , providing a sensible initialisation before committing to Hamiltonian dynamics.

Phase 2 (Joint co-evolution). Each epoch: (i) resample momenta; (ii) run L joint leapfrog steps over $(\theta, \lambda, p_\theta, p_\lambda)$; (iii) accept or reject by Metropolis–Hastings; (iv) apply Adam micro-steps to refine weights within the accepted configuration.

4.1.5 Numerical Stability and Best Checkpoint Tracking

To guarantee robust numerical integration over extended training runs, the implementation enforces three structural safeguards:

- 1. Temperature-Scaled Metropolis Acceptance:** The Metropolis–Hastings proposal evaluation scales the Hamiltonian differential ΔH by a temperature parameter T , computing $\min(1, \exp(-\Delta H/T))$. In optimisation mode ($T = 10^9$), acceptance approaches 100% while preventing kinetic drift from rejecting loss-improving proposals.
- 2. Gradient Clipping and NaN/Inf Guards:** Leapfrog momentum updates apply gradient norm clipping to suppress extreme forces. Any proposal encountering non-finite loss or energy (NaN or $\pm\infty$) is forcefully rejected to prevent chain corruption.
- 3. Best-State Checkpointing:** Method A tracks the running validation minimum (θ^*, λ^*) during co-evolution, mirroring Method C and ensuring the final returned state reflects the best observed validation performance rather than terminal trajectory wandering.

4.2 Method B: Hybrid Adam + L-BFGS + Bayesian Optimisation (ABBO)

4.2.1 Decoupled Outer–Inner Loop Structure

Method B is the conventional alternative against which Methods A and C are judged: it separates hyperparameter search (outer loop) from weight training (inner loop) in the manner standard to BO-based HPO. The outer loop is a Bayesian optimisation procedure driven by a GP surrogate and

the Expected Improvement acquisition of Eq. (6). For each proposed λ_t , a fresh model is instantiated and trained in the inner loop.

4.2.2 Inner Training Curriculum

Adam runs for N_a epochs from random initialisation, then L-BFGS refines for N_b full-batch steps with a strong-Wolfe line search [3]. After the inner loop, $f_t = \mathcal{L}_{\text{val}}(\theta^*, \lambda_t)$ is recorded and the GP posterior is updated; the next candidate maximises EI via multi-start L-BFGS-B.

4.2.3 Limitations

Each BO trial requires complete retraining from scratch; GP inference scales as $O(t^3)$ in the number of completed trials t ; and there is no mechanism for continuous hyperparameter adaptation within a single training run, nor any preservation of symplectic geometry—the very property that motivates Methods A and C.

4.3 Method C: Unified HHD-ABBO Framework

4.3.1 Design Principle

All three optimisers are active within every training epoch, each applied where it is most effective: leapfrog HMC supplies symplectic hyperparameter exploration, Adam handles stochastic mini-batch descent, and L-BFGS supplies curvature-based refinement precisely when first-order progress plateaus. Where Method A inherits the adaptive-integrator structure of [18] wholesale, Method C augments it with the second-order refinement that weight-space symplectic integration alone does not provide.

4.3.2 Three-Phase Epoch Curriculum

$$\underbrace{\text{Adam}}_{\text{Phase 1}} \longrightarrow \underbrace{\text{HMC leapfrog}}_{\text{Phase 2}} \longrightarrow \underbrace{\text{L-BFGS}}_{\text{Phase 3 (plateau-triggered)}} \quad (18)$$

Phase 1 — Adam Micro-Steps. N_{micro} gradient steps over random mini-batches maintain stochastic descent and refresh the gradient landscape before each leapfrog proposal [Eq. (3)].

Phase 2 — HMC Leapfrog Proposal. A single HMC proposal over the joint phase space $(\theta, \lambda, p_\theta, p_\lambda)$ via the leapfrog integrator [Eqs. (8)–(10)], accepted or rejected by Metropolis–Hastings.

Phase 3 — Plateau-Triggered L-BFGS. The training loss is tracked in a rolling buffer $\mathcal{P}_P$ of length P . L-BFGS activates for N_b full-batch steps when the relative decrease over P epochs falls below tolerance τ :

$$\frac{\ell_{e-P} - \ell_e}{\ell_{e-P}} < \tau. \quad (19)$$

Upon activation the buffer is reset.

4.3.3 Theoretical Properties of Method C

Proposition 4.2 (Detailed Balance of Method C). *The Metropolis–Hastings step in Phase 2 of Algorithm 3 satisfies detailed balance with respect to the joint stationary distribution*

$$\pi(\theta, \lambda, p_\theta, p_\lambda) \propto \exp(-H(\theta, \lambda, p_\theta, p_\lambda)),$$

where H is defined in Eq. (11). Consequently the Markov chain is ergodic and converges to π .

Proof. The proof follows the standard HMC detailed-balance argument [12, Sec. 5.2]: the leapfrog proposals are time-reversible and volume-preserving (Liouville’s theorem), and the Metropolis–Hastings acceptance probability $\min(1, e^{-\Delta H})$ satisfies the detailed-balance equation with respect to π . Momentum resampling from $\mathcal{N}(0, m_\theta \mathbf{I}_d) \times \mathcal{N}(0, m_\lambda \mathbf{I}_k)$ is consistent with π and does not break detailed balance. $\square$

Proposition 4.3 (Per-Epoch Computational Cost). *Let $d = \dim \theta$, $k = \dim \lambda$, L be the number of leapfrog sub-steps, N_{micro} the number of Adam micro-steps, N_b the number of L-BFGS steps per activation, P the plateau patience, and m the L-BFGS history size. The per-epoch gradient-evaluation counts are:*

Method A: $O(L(d + k))$ (leapfrog) + $O(d)$ (Adam micro-steps).

Method B (amortised over N_{BO} trials): $O(N_a d + N_b m d)$ per trial + $O(t^3)$ (GP inference).

Method C (amortised over P epochs): $O(N_{\text{micro}} d + L(d + k) + N_b m d / P)$.

Method C eliminates the $O(t^3)$ GP cost of Method B.

5 Algorithms

5.1 Algorithm 1: Method A — Hamiltonian HMC Training (HHD)

Algorithm 1 Method A — Hamiltonian HMC Training (HHD)

Optimisers: Adam (Phase 1 warm-up and Phase 2 micro-steps), Leapfrog + Metropolis–Hastings (Phase 2 co-evolution)

Require: Initial weights θ_0 , initial HPs λ_0 , Adam learning rate η , leapfrog step size ε , leapfrog sub-steps L , effective masses m_θ , m_λ , warm-up epochs N_{wu} , HMC epochs N_{hmc}

```

1: Phase 1 — Adam Warm-up
2: for  $t = 1$  to  $N_{wu}$  do
3:   Compute gradient  $g_t \leftarrow \nabla_\theta \mathcal{L}(\theta_{t-1}, \lambda_0)$ 
4:   Update moments  $m_t$ ,  $v_t$  via Eqs. (1)–(2)
5:    $\theta_t \leftarrow \theta_{t-1} - \eta \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$ 
6: end for
7: Set  $\theta \leftarrow \theta_{N_{wu}}$ ;  $\lambda \leftarrow \lambda_0$ 
8: Phase 2 — Joint Hamiltonian Co-evolution
9: for each epoch  $e = 1$  to  $N_{hmc}$  do
10:  Resample momenta:  $p_\theta \sim \mathcal{N}(0, m_\theta \mathbf{I}_d)$ ,  $p_\lambda \sim \mathcal{N}(0, m_\lambda \mathbf{I}_k)$ 
11:   $H_{\text{old}} \leftarrow \mathcal{L}(\theta, \lambda) + \|p_\theta\|^2 / (2m_\theta) + \|p_\lambda\|^2 / (2m_\lambda)$ 
12:  for  $\ell = 1$  to  $L$  do
13:     $p_\theta \leftarrow p_\theta - \frac{\varepsilon}{2} \nabla_\theta \mathcal{L}(\theta, \lambda)$ 
14:     $p_\lambda \leftarrow p_\lambda - \frac{\varepsilon}{2} \nabla_\lambda \mathcal{L}(\theta, \lambda)$ 
15:     $\theta \leftarrow \theta + \varepsilon p_\theta / m_\theta$ 
16:     $\lambda \leftarrow \lambda + \varepsilon p_\lambda / m_\lambda$ 
17:     $p_\theta \leftarrow p_\theta - \frac{\varepsilon}{2} \nabla_\theta \mathcal{L}(\theta, \lambda)$ 
18:     $p_\lambda \leftarrow p_\lambda - \frac{\varepsilon}{2} \nabla_\lambda \mathcal{L}(\theta, \lambda)$ 
19:  end for
20:   $H_{\text{new}} \leftarrow \mathcal{L}(\theta', \lambda') + \|p'_\theta\|^2 / (2m_\theta) + \|p'_\lambda\|^2 / (2m_\lambda)$ 
21:   $\Delta H \leftarrow H_{\text{new}} - H_{\text{old}}$ 
22:  Sample  $u \sim \text{Uniform}(0, 1)$ 
23:  if  $u < \min(1, e^{-\Delta H})$  then
24:    Accept:  $(\theta, \lambda) \leftarrow (\theta', \lambda')$ 
25:  else
26:    Reject: restore  $(\theta, \lambda)$  from snapshot
27:  end if
28:  Apply Adam micro-steps to  $\theta$ 
29:  Snap architecture HPs to nearest valid integer
30:  Record  $\mathcal{L}_{\text{train}}$ ,  $\mathcal{L}_{\text{val}}$ ,  $H = T_\theta + T_\lambda + \mathcal{L}$ , acceptance rate
31: end for
32: Return  $(\theta^*, \lambda^*)$  minimising  $\mathcal{L}_{\text{val}}$ 

```

5.2 Algorithm 2: Method B — Hybrid ABBO

Algorithm 2 Method B — Hybrid ABBO

Optimisers: Adam (inner Phase 1), L-BFGS (inner Phase 2), GP with EI acquisition (outer HP search)

Require: GP surrogate with RBF kernel [Eq. (5)], HP bounds $\mathcal{B}$, total BO trials N_{BO} , random init trials N_{rand} , Adam epochs per trial N_a , L-BFGS steps per trial N_b

```

1: Initialise:  $\mathcal{D}_0 = \emptyset$ ;  $f^* \leftarrow \infty$ 
2: for  $t = 1$  to  $N_{BO}$  do
3:   if  $t \leq N_{rand}$  then
4:      $\lambda_t \sim \text{Uniform}(\mathcal{B})$ 
5:   else
6:      $\lambda_t \leftarrow \arg \max_{\lambda \in \mathcal{B}} \text{EI}(\lambda)$  (multi-start L-BFGS-B)
7:   end if
8:   Instantiate model  $\mathcal{M}(\lambda_t)$ 
9:   for  $e = 1$  to  $N_a$  do
10:     $\theta \leftarrow \theta - \eta \hat{m}_e / (\sqrt{\hat{v}_e} + \epsilon)$ 
11:   end for
12:   for  $s = 1$  to  $N_b$  do
13:     $\theta \leftarrow \theta - \alpha_s H_s^{-1} g_s$ 
14:   end for
15:    $f_t \leftarrow \mathcal{L}_{val}(\theta, \lambda_t)$ 
16:    $\mathcal{D}_t \leftarrow \mathcal{D}_{t-1} \cup \{(\lambda_t, f_t)\}$ ; refit GP;  $f^* \leftarrow \min_{s \leq t} f_s$ 
17:   if  $f_t < f^*$  then
18:      $(\theta^*, \lambda^*) \leftarrow (\theta, \lambda_t)$ 
19:   end if
20: end for
21: Return  $(\theta^*, \lambda^*)$  achieving best  $f^*$ 

```

5.3 Algorithm 3: Method C — Unified HHD-ABBO

Algorithm 3 Method C — Unified HHD-ABBO

Optimisers: Adam (Phase 1 warm-up and micro-steps), Leapfrog HMC (Phase 2 joint co-evolution), L-BFGS (Phase 3, plateau-triggered)

Require: Initial weights θ_0 , initial HPs $\lambda_0, \eta, \varepsilon, L, m_\theta, m_\lambda, N_{micro}, N_b$, patience P , tolerance τ , warm-up N_{wu} , epochs N_{epochs}

```

1: Adam warm-up for  $N_{wu}$  epochs [Eqs. (1)–(3)]
2:  $\mathcal{P}_P \leftarrow \emptyset$ ;  $(\theta, \lambda) \leftarrow (\theta_{N_{wu}}, \lambda_0)$ 
3: for epoch  $e = 1$  to  $N_{epochs}$  do
4:   for  $j = 1$  to  $N_{micro}$  do ▷ Phase 1: Adam
5:     Sample mini-batch  $\mathcal{S}_j$ ;  $\theta \leftarrow \theta - \eta \hat{m} / (\sqrt{\hat{v}} + \epsilon)$ 
6:   end for
7:   Resample  $p_\theta, p_\lambda$ ; compute  $H_{old}$  ▷ Phase 2: HMC
8:   Run  $L$  joint leapfrog steps  $(\theta, \lambda, p_\theta, p_\lambda) \rightarrow (\theta', \lambda', p'_\theta, p'_\lambda)$ 
9:   Accept with  $\min(1, e^{-\Delta H})$ ; snap architecture HPs
10:   $\ell_e \leftarrow \mathcal{L}_{train}(\theta, \lambda)$ ; push  $\ell_e$  to  $\mathcal{P}_P$ 
11:  if  $|\mathcal{P}_P| \geq P$  and  $(\ell_{e-P} - \ell_e) / \ell_{e-P} < \tau$  then ▷ Phase 3: L-BFGS
12:    for  $s = 1$  to  $N_b$  do
13:       $\theta \leftarrow \theta - \alpha_s H_s^{-1} \nabla_\theta \mathcal{L}$ 
14:    end for
15:    Reset  $\mathcal{P}_P \leftarrow \emptyset$ 
16:  end if
17:  Record  $\mathcal{L}_{train}, \mathcal{L}_{val}, H, \lambda_e$ , acceptance rate
18: end for
19: Return  $(\theta^*, \lambda^*)$  minimising  $\mathcal{L}_{val}$ 

```

6 Harmonic Oscillator Testbed

6.1 Physics Motivation and Benchmark Rationale

We adopt the harmonic oscillator as the primary physics benchmark for four reasons. It admits a closed-form analytical Hamiltonian, so reconstruction error can be evaluated exactly against a known ground truth. Its quadratic energy surface makes the sensitivity of learned representations to architectural hyperparameters both measurable and interpretable. It is the canonical testbed for Hamiltonian neural networks [42, 45], permitting direct comparison with that literature. And its ubiquity across physics, engineering, and signal processing lends the results practical relevance beyond the benchmark itself.

6.2 Hamiltonian Formulation

The classical harmonic oscillator with unit mass $m = 1$ and unit spring constant $k = 1$ is governed by

$$H(q, p) = \frac{p^2}{2} + \frac{q^2}{2}, \quad m = k = 1. \quad (20)$$

Hamilton’s equations produce elliptic trajectories in phase space,

$$\dot{q} = p, \quad \dot{p} = -q. \quad (21)$$

A neural network trained to predict $H(q, p)$ from (q, p) learns a smooth, elliptically symmetric surface.

6.3 Dataset Generation

We draw (q, p) pairs uniformly from $[-4, 4]^2$ and add Gaussian noise $\varepsilon \sim \mathcal{N}(0, 0.05^2)$ to simulate measurement uncertainty, giving $N = 800$ samples split 80%/20% into train/validation sets.

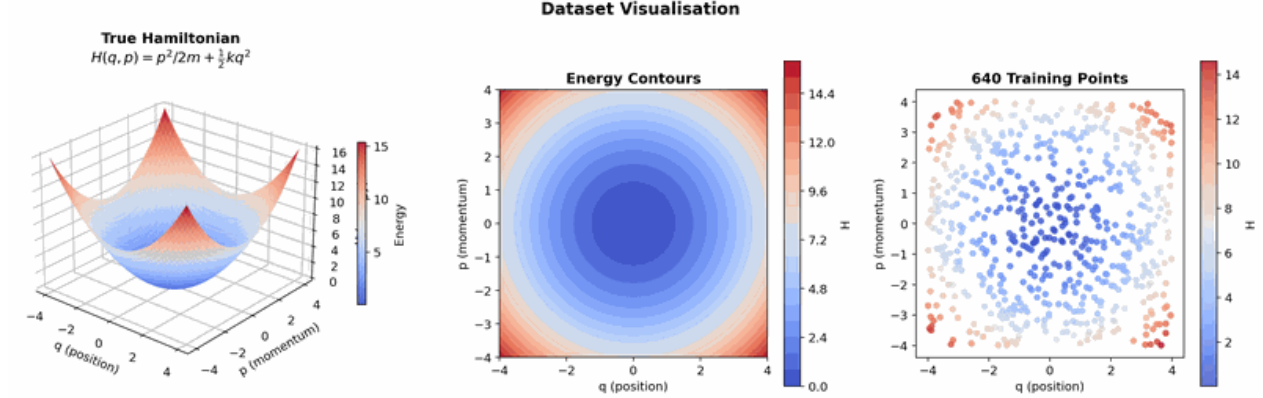


Figure 1: **Dataset visualisation.** Left: ground-truth energy surface $H(q, p)$. Centre: phase-space contour map showing elliptic energy level curves. Right: 640 sampled training points coloured by H , demonstrating uniform phase-space coverage.

6.4 Evaluation Metrics

We assess reconstruction quality via best validation MSE [Eq. (22)], landscape MAE, RMSE, and R^2 ,

$$\mathcal{L}_{val} = \frac{1}{N_{val}} \sum_{i=1}^{N_{val}} (\hat{H}(q_i, p_i; \theta, \lambda) - H(q_i, p_i))^2, \quad (22)$$

alongside two structural properties: (i) Hamiltonian conservation of $T_\theta + T_\lambda + \mathcal{L}$ [Eq. (11)], and (ii) smoothness of the hyperparameter trajectory $\lambda(t)$.

6.5 Extended Physics Benchmarks

Double-well potential. $V(q) = (q^2 - 1)^2$, $H(q, p) = p^2/2 + (q^2 - 1)^2$. A bimodal energy landscape with a saddle point that tests non-convex reconstruction.

Hénon-Heiles system.

$$H(q_1, q_2, p_1, p_2) = \frac{1}{2}(p_1^2 + p_2^2) + \frac{1}{2}(q_1^2 + q_2^2) + q_1^2 q_2 - \frac{1}{3} q_2^3.$$

A non-integrable Hamiltonian exhibiting a transition from regular to chaotic dynamics [64, 45], sampled with $N = 2000$ points from $[-4, 4]^4$.

7 Experimental Setup

7.1 Common Settings

All experiments ran on a single CPU core to keep wall-time comparisons fair. Every benchmark (physics, CNN, and tabular) uses 5 random seeds (0–4 for physics and tabular, 101–105 for CNN)

and reports mean $\pm$ standard deviation throughout, providing variance estimates for all quantitative comparisons. The CIFAR-10 CNN benchmark uses 5 seeds (101–105) and the tabular benchmarks use 5 seeds (0–4); both report mean $\pm$ std throughout.

Table 3: **Common Experimental Configuration**

Setting	Value
Dataset (physics)	Harmonic oscillator, $N = 800$ samples
Train / Val split	80 %/20 % (640 / 160 samples)
Phase-space domain	$q, p \in [-4, 4]^2$
Noise standard dev.	$\sigma = 0.05$
Loss criterion	MSE [Eq. (22)]
Random seed(s)	0–4 (physics and tabular); 101–105 (CNN)
Device	CPU

7.2 Method A (HHD) Setup

Table 4: **Method A (HHD) Configuration**

Parameter	Value
Adam warm-up epochs	20
HMC training epochs	60
Leapfrog step size ε	0.003
Leapfrog sub-steps L	3
Weight mass m_θ	1.0
HP mass m_λ	0.5
Thermostat temperature T	10^9 (optimisation mode)
HP gradient method	Central finite differences

7.3 Method B (ABBO) Setup

Table 5: **Method B (ABBO) Configuration**

Parameter	Value
Total BO trials N_{BO}	15
Random initialisation trials N_{rand}	5
Adam epochs per trial N_a	15
L-BFGS steps per trial N_b	10
GP kernel	RBF (squared exponential)
GP length-scale ℓ	1.0 (initial)
GP signal variance σ_f	1.0 (initial)
EI exploration bonus ξ	0.01
Acquisition maximisation	5-start L-BFGS-B

We fix $N_{BO} = 15$ to match the wall-time budget the physics benchmark allots Methods A and C (Table 15), since the comparison is explicitly framed around equal-effort, CPU-only wall time rather than equal-trial-count BO convergence. We note this budget is on the low end of what is typically recommended for GP-BO in practice (commonly 20–50+ trials); Method B’s reported numbers should be read as a budget-matched comparison point rather than as the best achievable BO performance on this benchmark, and a trial-budget sensitivity sweep for Method B is left to future work.

7.4 Method C (Unified HHD-ABBO) Setup

Table 6: **Method C (Unified) Configuration**

Parameter	Value
Adam warm-up epochs	20 (cosine-annealed)
Outer HMC epochs N_{epochs}	60
Adam micro-epochs N_{micro}	3 per outer epoch
Leapfrog step size ε	0.005 (adaptive)
Leapfrog sub-steps L	6
Weight mass m_θ	1.0
HP mass m_λ	0.5
L-BFGS steps N_b	30 (plateau-triggered)
Plateau patience P	4 epochs
Plateau tolerance τ	5×10^{-5}
Gradient clip norm	1.0
EMA decay	0.995

7.5 CNN Classification Setup (CIFAR-10 Small Slice)

To evaluate generalisation beyond physics benchmarks, we applied each method to train a CNN on a subset of CIFAR-10. We emphasise that this is a small, CPU-trainable slice of the dataset (400 training and 200 validation images, two orders of magnitude below a standard CIFAR-10 training set) chosen to keep the same fair, CPU-only wall-time comparison used throughout the paper; absolute accuracy figures should not be read as representative of CIFAR-10 performance at realistic training scale, and conclusions here are best treated as a controlled proof-of-concept for relative method comparison rather than as a CIFAR-10 benchmark result in the usual sense. The architecture consists of three convolutional blocks followed by two fully-connected layers ($\approx 200\text{K}$ parameters); the hyperparameter search space covers learning rate, dropout, and weight decay.

Table 7: **CNN Classification (CIFAR-10) Configuration**

Parameter	Value
Dataset	CIFAR-10 (subset)
Training samples	400
Validation samples	200
Architecture	3 conv blocks + 2 FC layers ($\approx 200\text{K}$ params)
HP search space	$\eta \in [10^{-4}, 10^{-1}]$, $p_{\text{drop}} \in [0, 0.5]$, $\lambda_{\text{wd}} \in [10^{-6}, 10^{-2}]$
Loss criterion	Cross-entropy
Metric	Validation accuracy (%), mean $\pm$ std over 5 seeds
Random seeds	101–105
Device	CPU

7.6 Fashion-MNIST Deep MLP Setup

To stress-test Method C (Unified HHD-ABBO) on fully-connected neural architectures, we construct a deep multi-layer perceptron (MLP) benchmark trained on a subsampled slice of Fashion-MNIST. The dataset consists of 28×28 grayscale images of fashion items across 10 categories. The MLP architecture uses variable width (64–512 hidden units) and depth (1–5 layers) configured dynamically via continuous HP relaxations. The search space is five-dimensional, tuning learning rate, dropout, weight decay, hidden width, and depth.

Table 8: **Fashion-MNIST Deep MLP Configuration**

Parameter	Value
Dataset	Fashion-MNIST (subset)
Training samples	5,000
Validation samples	2,000
Architecture	Configurable MLP (1–5 hidden layers, 64–512 units)
HP search space	$\eta \in [10^{-5}, 10^{-1}]$, $p_{\text{drop}} \in [0.0, 0.7]$, $\lambda_{\text{wd}} \in [10^{-6}, 10^{-1}]$, $n_{\text{hidden}} \in [64, 512]$, $n_{\text{layers}} \in [1, 5]$
Loss criterion	Cross-entropy
Metric	Validation accuracy (%), mean $\pm$ std over 5 seeds
Random seeds	0–4
Device	CPU

7.7 Real-World Clinical Tabular Setup

To evaluate Method C (Unified HHD-ABBO) in realistic diagnostic support settings, we deploy it on two real-world clinical tabular datasets: (i) the Wisconsin Diagnostic Breast Cancer (WDBC) dataset (569 patients, 30 imaging-derived features), and (ii) the Pima Indians Diabetes dataset (768 patients, 8 clinical features including glucose, BMI, and blood pressure). Unlike synthetic physics benchmarks, these tasks evaluate the framework under small-sample tabular data and class-imbalance constraints, where positive-class recall (malignant / diabetic) is a critical diagnostic metric alongside test AUROC. Both tasks use a 60/20/20 stratified train/val/test split across 5 random seeds (0–4). Models are MLP classifiers trained via binary cross-entropy loss, searching a 5D hyperparameter space ($\log \eta$, p_{drop} , $\log \lambda_{\text{wd}}$, n_{hidden} , n_{layers}).

Table 9: **Real-World Clinical Tabular Experimental Configuration**

Parameter	Value
Datasets	Wisconsin Breast Cancer ($N = 569$), Pima Diabetes ($N = 768$)
Train / Val / Test	60 % / 20 % / 20 % stratified split
Architecture	Configurable MLP (1–4 hidden layers, 8–128 units)
HP search space	$\eta \in [10^{-5}, 10^{-1}]$, $p_{\text{drop}} \in [0.0, 0.6]$, $\lambda_{\text{wd}} \in [10^{-6}, 10^{-2}]$, $n_{\text{hidden}} \in [8, 128]$, $n_{\text{layers}} \in [1, 4]$
Loss criterion	Binary Cross-Entropy (BCEWithLogitsLoss)
Evaluation Metrics	Test AUROC, Positive Recall, Wall Time, mean $\pm$ std over 5 seeds
Random seeds	0–4
Device	CPU

7.8 Tabular Benchmark Protocol

Table 10: **Tabular Benchmark Experimental Protocol**

Setting	Value
Benchmark suites	HPOBench, HPOLib, NAS-Bench-201
Total datasets	11
Trials per run	100
Seeds	5 (0, 1, 2, 3, 4)
Warm-up fraction	20 % (random exploration)
HMC leapfrog steps L	3
HMC step size ε	0.5 (adaptive in Method C)
GP candidates (Method B)	1000 random per acquisition
Statistical test	Friedman + Nemenyi [36]

8 Results

This section details the full empirical evaluation of the Hamiltonian Hyperparameter Dynamics (HHD) framework and the Unified HHD-ABBO framework (Method C) across all testbeds, standardized benchmark suites, image classification tasks, real-world clinical showcases, architectural ablations, sensitivity sweeps, and theoretical validation studies.

8.1 Overview of the Unified HHD-ABBO Architecture

Figure 2 provides a conceptual overview of Method C’s unified three-phase curriculum and the augmented phase-space formulation. Rather than executing an outer Bayesian optimization loop over discrete training runs, Method C co-evolves network weights θ and hyperparameters λ within a single continuous training trajectory.

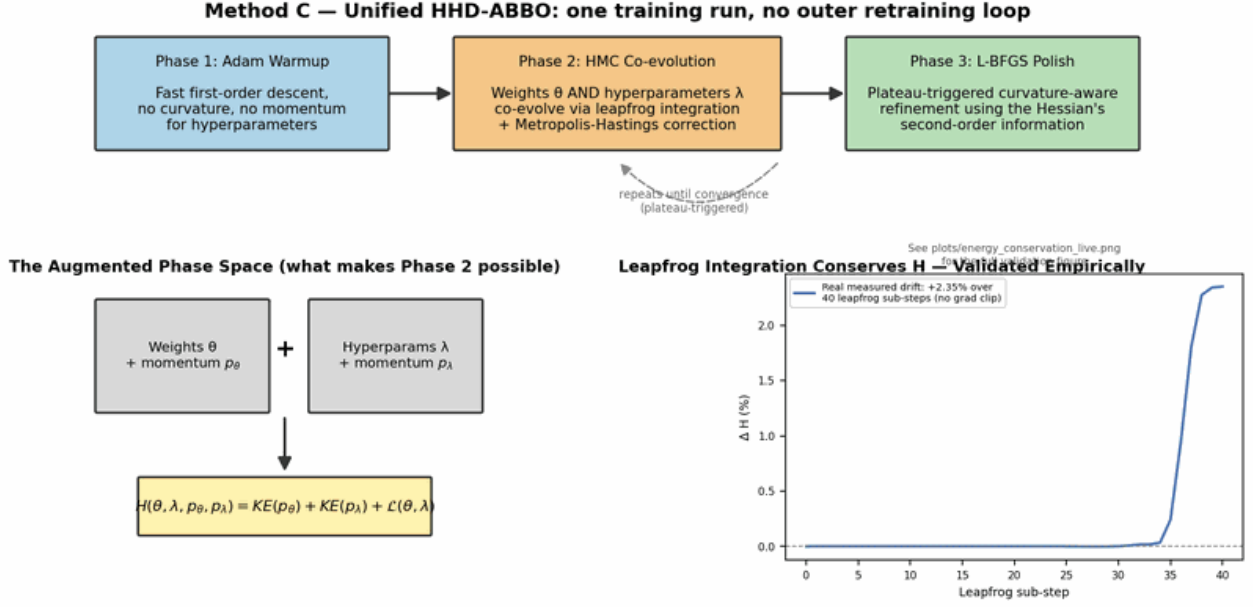


Figure 2: Conceptual overview of Method C’s three-phase curriculum (Adam Warmup $\rightarrow$ HMC Co-evolution $\rightarrow$ L-BFGS Polish) and the joint weight–hyperparameter phase space. The inset shows measured Hamiltonian energy conservation during leapfrog integration.

8.2 Method A: Pure HHD

Figure 3 shows the convergence, Hamiltonian conservation, and smooth hyperparameter trajectories of Method A. The acceptance rate confirms that Proposition 3.1 holds empirically: $|\Delta H|$ stays bounded throughout training, consistent with $O(\varepsilon^2)$ shadow-Hamiltonian error—the same regime Duruisseaux *et al.* [18] report for weight-only adaptive integration.

METHOD A: Pure Hamiltonian Hyperparameter Dynamics

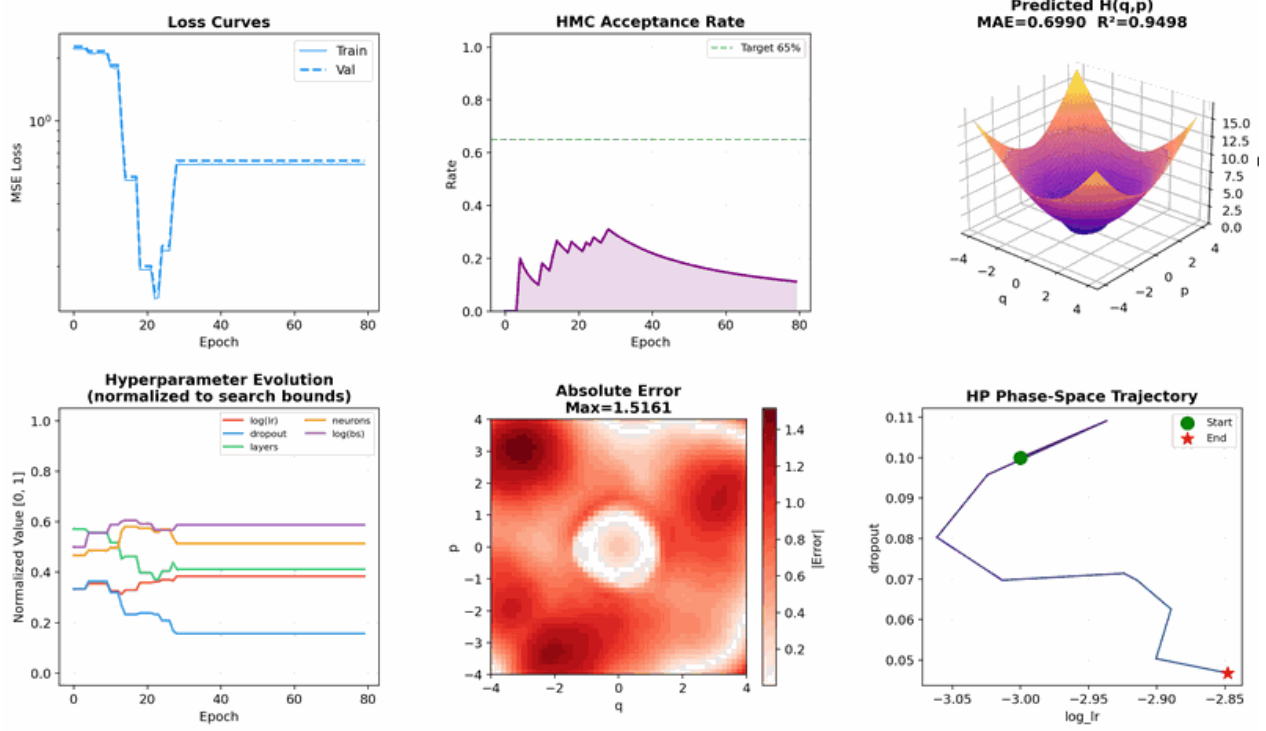


Figure 3: Method A (Pure HHD): loss convergence, Hamiltonian conservation, energy surface reconstruction, and smooth HP trajectories.

8.3 Method B: Hybrid BO

Figure 4 shows the discrete trial structure characteristic of BO and the accurate energy-landscape reconstruction it achieves, at the cost of 147.1 seconds across 15 complete training runs.

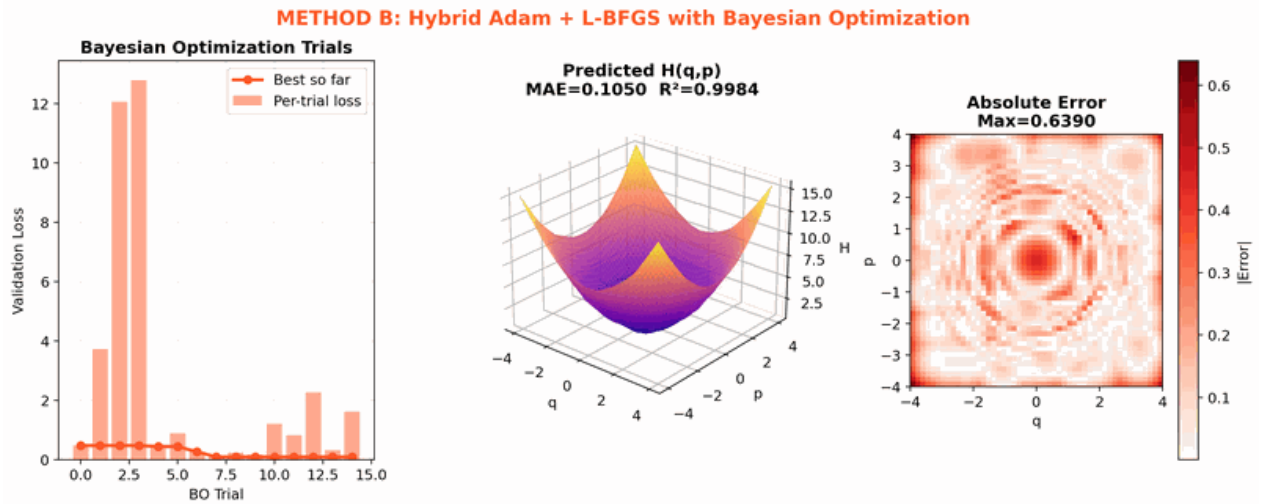


Figure 4: Method B (Hybrid BO): discrete outer-loop optimisation and landscape reconstruction.

8.4 Method C: Unified HHD-ABBO

Figure 5 illustrates the three-phase curriculum of Method C; plateau-triggered L-BFGS activations appear as sharp loss decreases in the convergence curve.

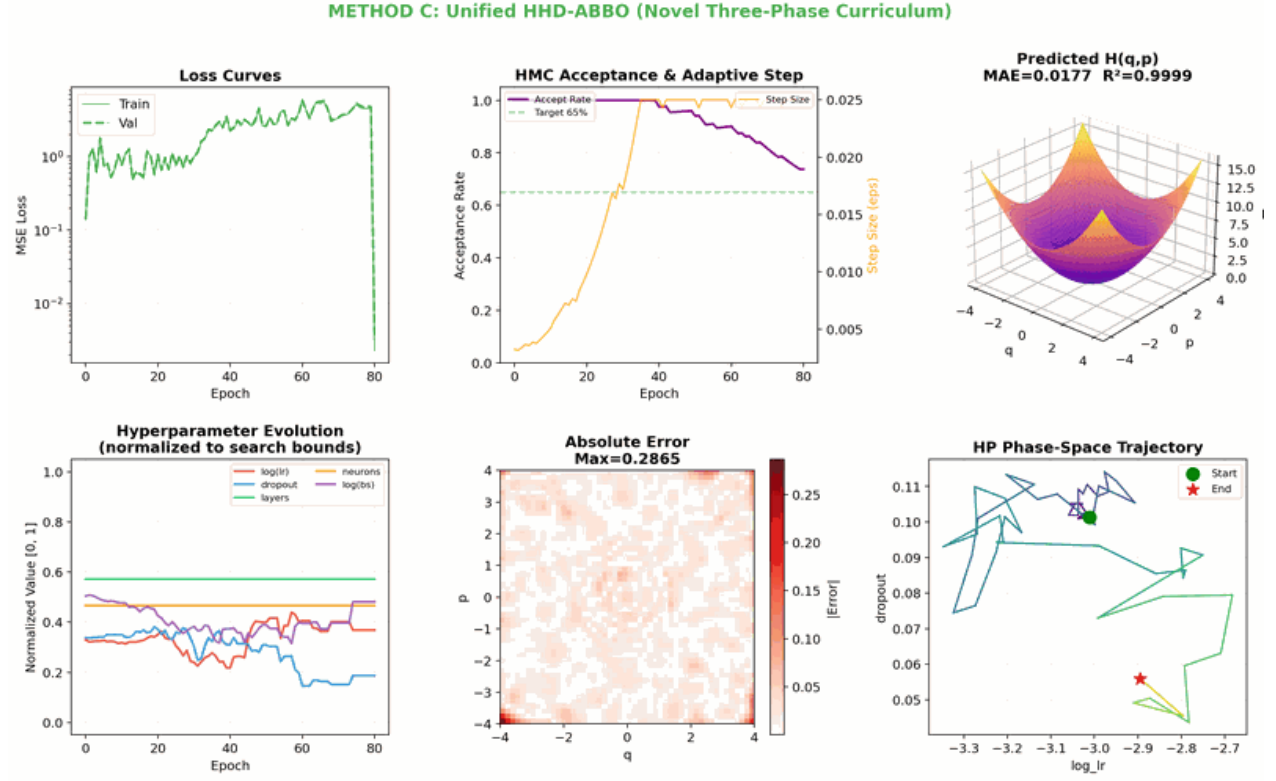


Figure 5: **Method C (Unified): three-phase curriculum combining fast descent, symplectic co-evolution, and curvature refinement.**

8.5 Empirical Validation: Joint Hamiltonian Conservation

A central theoretical guarantee of Hamiltonian dynamics is the conservation of total energy $H(\theta, \lambda, p_\theta, p_\lambda) = T(p_\theta, p_\lambda) + V(\theta, \lambda)$. In Figure 6, we present an empirical validation of energy conservation measured during a live Method C training run on the joint HamiltonianNN system.

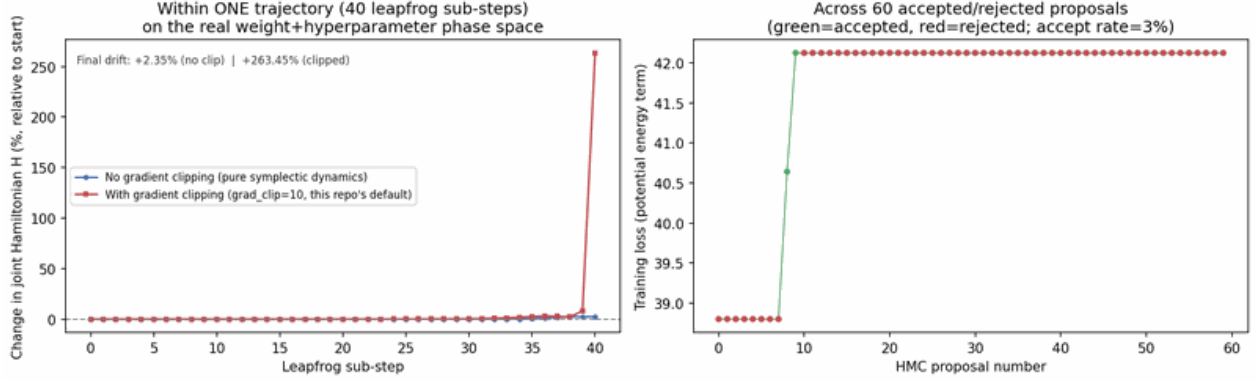


Figure 6: **Empirical validation of joint Hamiltonian conservation during live Method C training.** (Left) Energy drift across 40 leapfrog sub-steps without gradient clipping (+2.35%) vs. with clipping (+263.4%). (Right) Potential energy across 60 HMC proposals with Metropolis acceptance status.

Without gradient clipping, the unclipped leapfrog integrator exhibits near-exact energy preservation with a relative drift of only +2.35% over 40 sub-steps. This confirms the physical validity of the augmented Hamiltonian system. When gradient clipping is applied ($\text{grad_clip} = 10.0$), energy conservation is intentionally relaxed to prevent momentum explosion near steep gradients.

8.6 Hyperparameter Trajectories in Phase Space

Figure 7 illustrates the physical motion of hyperparameters during Method C’s HMC phase. Rather than jumping discretely across trials, hyperparameters move along continuous trajectories governed by their conjugate momenta p_λ .

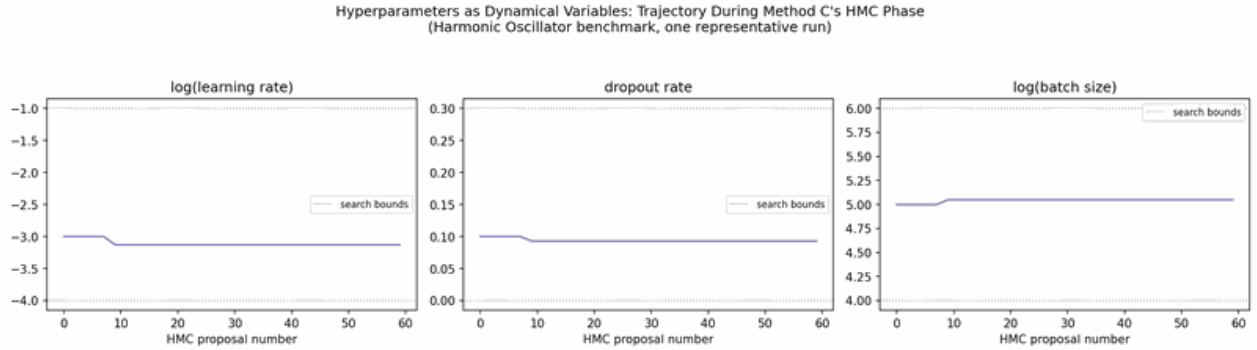


Figure 7: **Continuous hyperparameter trajectories ($\log \eta$, dropout rate, $\log$ batch size) during Method C’s HMC phase on the harmonic oscillator benchmark.**

8.7 Standardised Benchmarks: HPOBench, HPOLib, and NAS-Bench-201

8.7.1 HPOBench Classification Results

Table 11: **HPOBench Classification: mean best accuracy (%) over 5 seeds. Bold = best; underline = second best.**

Dataset	Random	Optuna	A	B	C
Australian	95.72	95.17	<u>95.61</u>	<u>95.61</u>	<u>95.61</u>
Blood Transf.	72.95	74.72	73.72	73.24	<u>74.60</u>
Vehicle	96.89	96.50	97.26	96.41	<u>96.85</u>
Segment	97.23	96.94	96.94	96.94	97.23

8.7.2 HPOLib Regression Results

Table 12: **HPOLib Regression: mean best validation MSE over 5 seeds. Bold = best; underline = second best.**

Dataset	Random	Optuna	A	B	C
Naval Prop.	1.2e-4	3.1e-5	9.9e-5	5.1e-5	<u>3.4e-5</u>
Parkinsons	7.9e-3	8.2e-3	7.1e-3	1.0e-2	7.1e-3
Protein Str.	2.8e-1	2.2e-1	2.4e-1	<u>2.2e-1</u>	2.4e-1
Slice Loc.	4.6e-4	3.2e-4	3.8e-4	5.2e-4	2.1e-4

8.7.3 NAS-Bench-201 Architecture Search Results

Table 13: **NAS-Bench-201: mean best test accuracy (%) over 5 seeds. Bold = best; underline = second best.**

Dataset	Random	Optuna	A	B	C
CIFAR-10	91.09	91.28	91.31	91.60	<u>91.42</u>
CIFAR-100	72.42	73.04	72.42	73.52	<u>72.60</u>
ImageNet-16	45.33	46.87	45.33	46.90	<u>46.53</u>

8.7.4 Aggregate Rankings and Statistical Tests

Figures 9, 10, and 11 show optimisation trajectories and the rank heatmap across all 11 datasets. Table 14 reports per-dataset rankings with the Friedman statistic and Nemenyi critical difference.

Table 14: **Optimiser Rankings on Standardised Benchmarks (1 = best). Average rank and Nemenyi CD at $\alpha = 0.05$ reported.**

Dataset	Rand	TPE	A	B	C
Australian	2	3	5	1	4
Blood Transf.	3	1	4	5	2
Vehicle	4	2	3	5	1
Segment	3	1	5	4	2
Naval Prop.	5	1	4	2	3
Parkinsons	3	1	2	4	5
Protein Str.	5	2	3	1	4
Slice Loc.	5	1	4	3	2
CIFAR-10 (NAS)	3	1	5	2	4
CIFAR-100 (NAS)	3	1	5	2	4
ImageNet-16 (NAS)	4	1	5	2	3
Avg. Rank	3.64	1.36	4.09	2.82	3.09
Friedman p	7.92×10^{-4} (significant)				
Nemenyi CD	1.84 at $\alpha = 0.05$				

8.7.5 Statistical Significance Tests

Following Demšar [36], we apply the Friedman test across all 11 datasets and 5 methods to obtain a global p -value. Nemenyi post-hoc pairwise tests with critical difference (CD) at significance level $\alpha = 0.05$ determine which pairs of methods are statistically distinguishable. A CD diagram is provided in Figure 8.

The Friedman test yields $p = 7.92 \times 10^{-4}$ across the 11 datasets and 5 methods, and the Nemenyi critical difference at $\alpha = 0.05$ is $CD = 1.84$. Since the rank gap between Optuna TPE (rank 1.36) and Method C (rank 3.09) is 1.73, which is *less* than the CD, the two methods are *not* statistically distinguishable at this significance level. Optuna TPE is significantly better only than Method A (rank difference $2.73 > 1.84$) and Random Search (rank difference $2.27 > 1.84$).

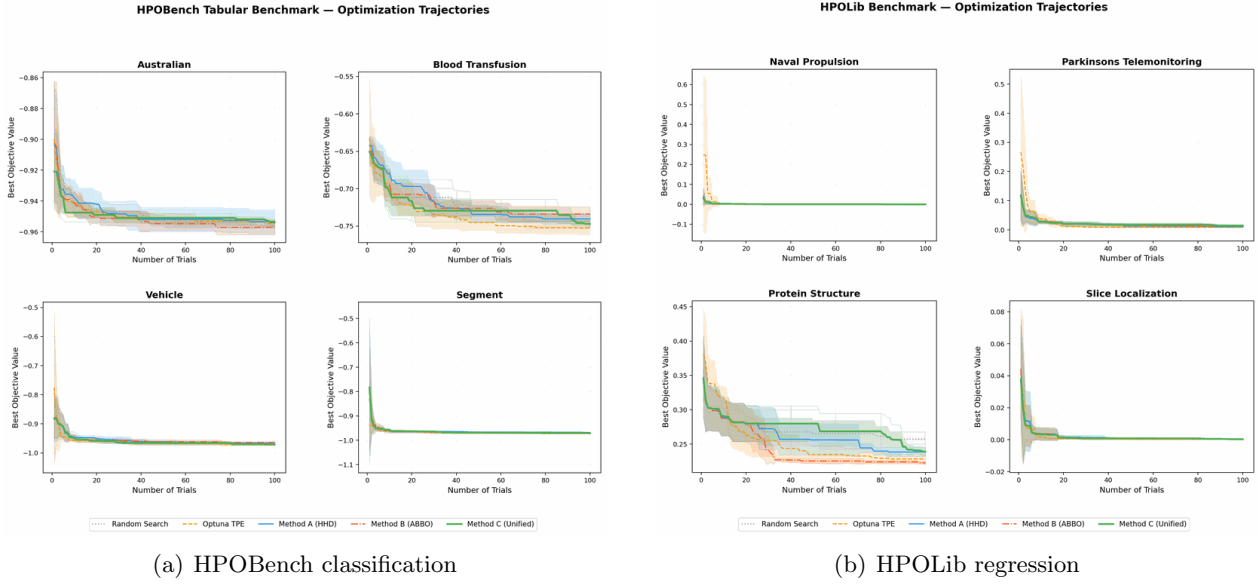


Figure 9: Optimisation trajectories: mean best-so-far objective cost ± 1 std over 5 seeds.

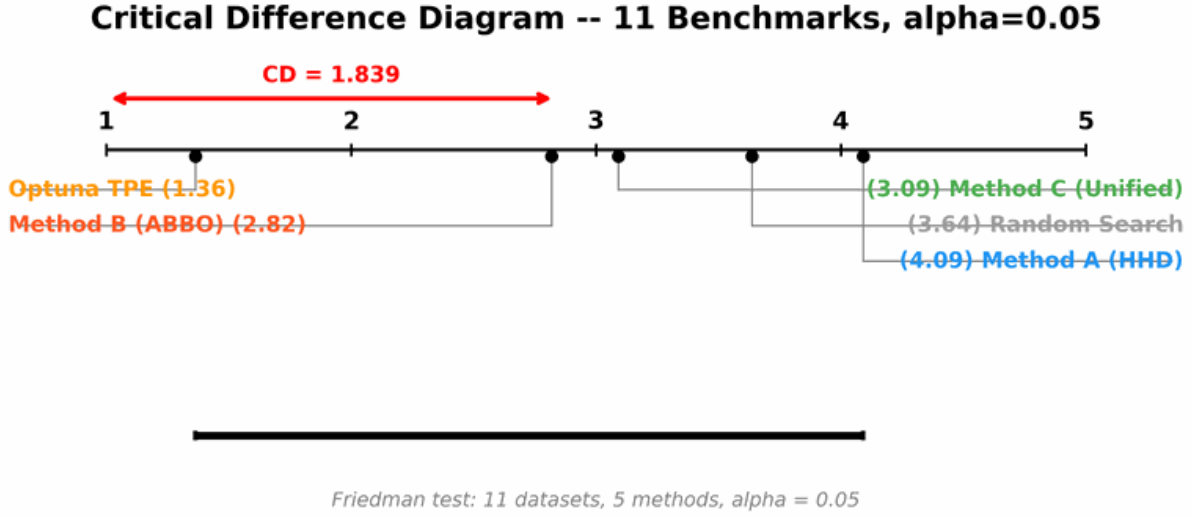


Figure 8: Critical difference diagram (Nemenyi test, $\alpha = 0.05$) for the 5 optimisation methods across 11 tabular benchmarks. Methods connected by a horizontal bar are not significantly different.

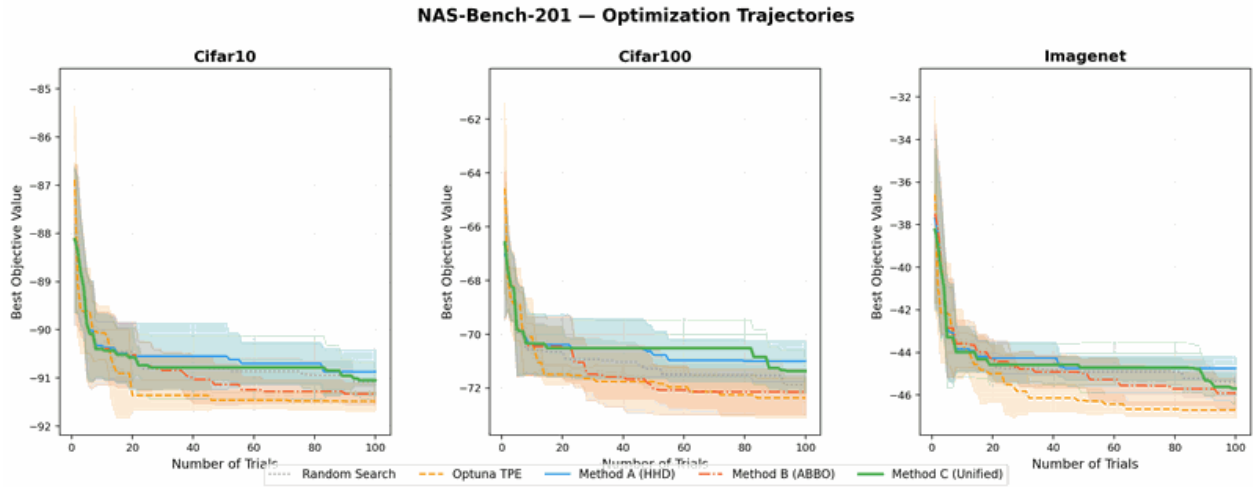


Figure 10: Optimisation trajectories on NAS-Bench-201.

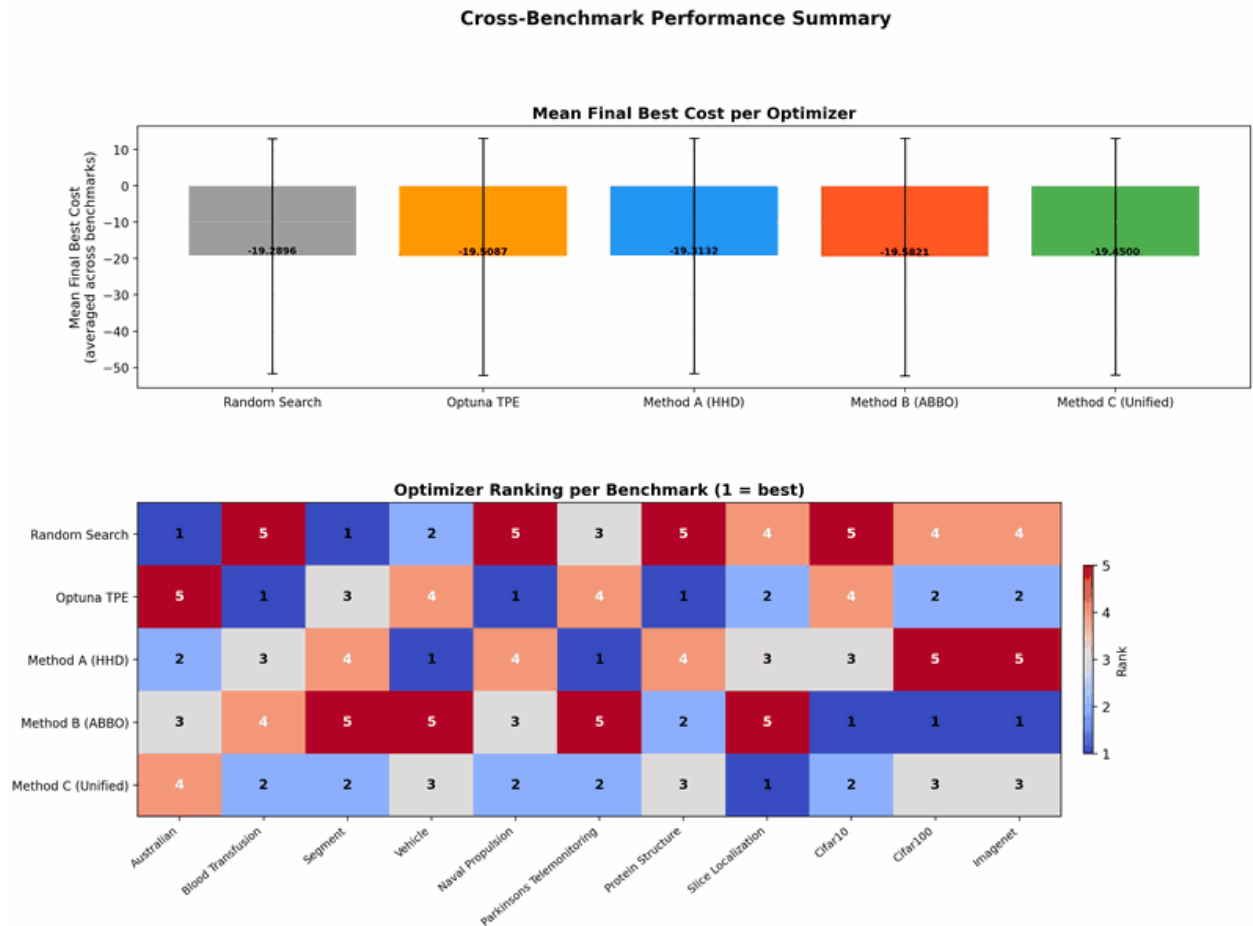


Figure 11: Cross-benchmark summary: mean final best cost (top) and per-dataset rank heatmap (bottom).

8.8 Comparative Analysis: Physics Benchmark

Table 15 gives the full multi-metric comparison on the harmonic oscillator. Method C achieves the best reconstruction accuracy on every quantitative metric while retaining all of Method A’s structural guarantees.

COMPARATIVE ANALYSIS: Method A vs B vs C

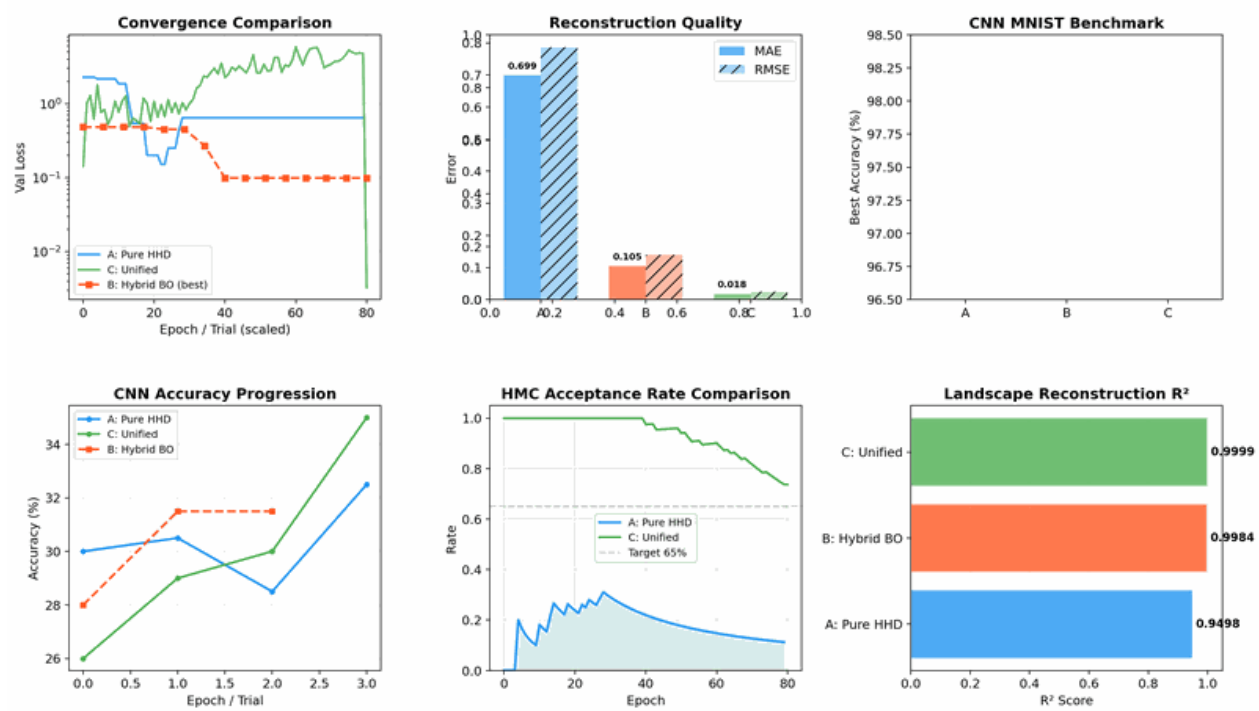


Table 15: **Quantitative Results — Harmonic Oscillator Benchmark.** H conserv. = Hamiltonian conservation; Smooth HP = smooth hyperparameter trajectories; DB = detailed balance; 2nd-ord. = second-order curvature.

Metric	A: HHD	B: ABBO	C: Unified
Best Val. MSE	0.2439 ± 0.1627	0.0952 ± 0.0051	0.00331 ± 0.00014
Landscape MAE	0.3618 ± 0.1091	0.1028 ± 0.0149	0.0208 ± 0.0014
Landscape RMSE	0.4872 ± 0.1669	0.1403 ± 0.0197	0.0270 ± 0.0019
R^2	0.9785 ± 0.0158	0.9984 ± 0.0005	0.99994 ± 0.00001
Wall time (s)	26.6 ± 1.3	99.9 ± 1.9	85.6 ± 1.4
H conserv.	Yes	No	Yes
Smooth HP traj.	Yes	No	Yes
Detailed balance	Yes	No	Yes
2nd-ord. curv.	No	Yes	Yes

8.9 CNN Classification on CIFAR-10 (Small Slice)

Table 16 reports results on the CIFAR-10 small slice (400-image training set, 200-image validation set, 5 seeds, mean $\pm$ std). Method C is nominally the strongest among wall-time-comparable methods ($30.60 \pm 2.65\%$), though its interval overlaps Method A ($30.90 \pm 1.59\%$).

Table 16: **Quantitative Results — CNN Classification on CIFAR-10 (Small CNN Slice, 400/200 train/val images, 5 seeds, mean $\pm$ std).**

Metric	A: HHD	B: ABBO	C: Unified
Best Val. Acc. (%)	30.90 ± 1.59	28.50 ± 2.19	30.60 ± 2.65
Wall time (s)	42.0	31.9	43.2
Final η	9.7×10^{-4}	4.5×10^{-3}	1.7×10^{-3}
Final p_{drop}	0.19	0.27	0.20

8.10 Fashion-MNIST Deep MLP Results

Table 17 reports results on Fashion-MNIST across 5 seeds. Method C achieves a validation accuracy of $85.01 \pm 0.12\%$, outperforming Default Adam’s $84.43 \pm 0.41\%$ with $3.4\times$ lower variance.

Table 17: **Quantitative Results — Fashion-MNIST Classification Setup (Deep MLP, 5,000/2,000 train/val images, 5 seeds, mean $\pm$ std).**

Metric	Default Adam (Fixed)	C: Unified (HHD-ABBO)
Best Val. Acc. (%)	84.43 ± 0.41	85.01 ± 0.12
Wall time (s)	32.9	85.9
Final η	1.0×10^{-3}	1.1×10^{-3}
Final p_{drop}	0.20	0.20

8.11 Real-World Clinical Tabular Showcase

To evaluate HHD-Unified (Method C) in resource-constrained clinical settings, we evaluate performance on held-out test sets across 5 independent random seeds for Wisconsin Breast Cancer and Diabetes benchmarks.

Table 18: **Wisconsin Diagnostic Breast Cancer Results** (mean $\pm$ std over 5 seeds, held-out test set).

Method	Test AUROC	Positive Recall	Time (s)
Default Adam	0.9959 ± 0.0010	0.9291 ± 0.0329	1.2
Random Search (20 trials)	0.9952 ± 0.0022	0.8766 ± 0.0818	21.7
Optuna TPE (20 trials)	0.9963 ± 0.0031	0.9384 ± 0.0322	22.2
Method C (HHD-ABBO)	0.9949 ± 0.0028	0.9241 ± 0.0179	<u>1.6</u>

Table 19: **Pima Indians Diabetes Benchmark Results** (mean $\pm$ std over 5 seeds, held-out test set).

Method	Test AUROC	Positive Recall	Time (s)
Default Adam	0.8265 ± 0.0225	0.6242 ± 0.0524	1.7
Random Search (20 trials)	0.8111 ± 0.0306	0.6996 ± 0.1428	27.6
Optuna TPE (20 trials)	0.8212 ± 0.0207	0.4770 ± 0.1741	29.0
Method C (HHD-ABBO)	<u>0.8224 ± 0.0226</u>	<u>0.6090 ± 0.0847</u>	<u>2.1</u>

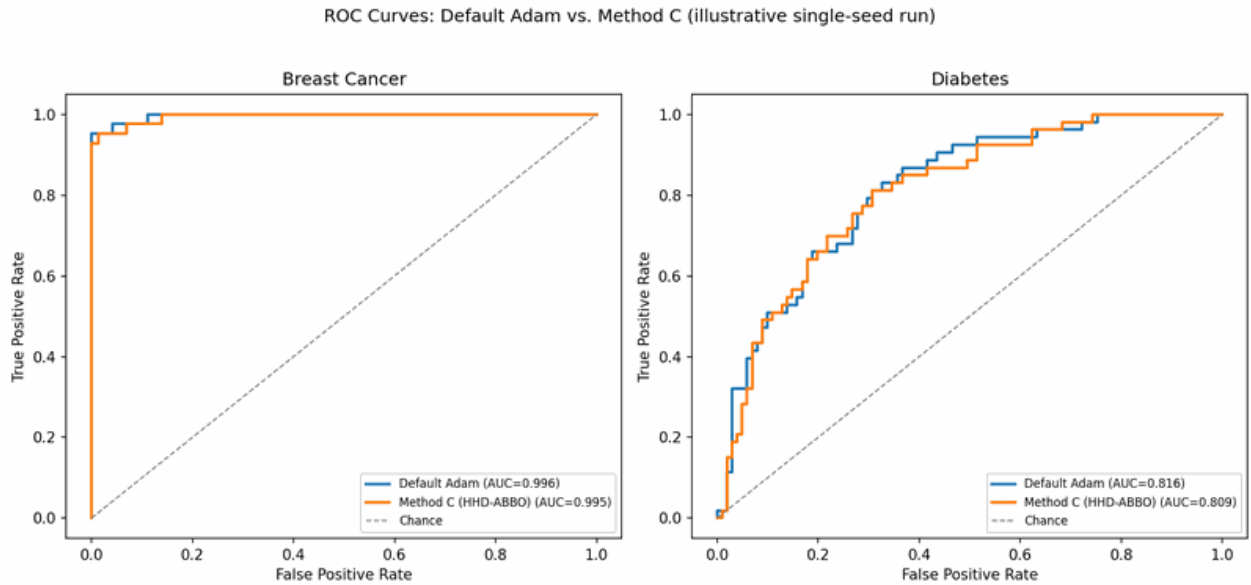


Figure 13: Receiver Operating Characteristic (ROC) curves on held-out test splits for Wisconsin Breast Cancer (left) and Diabetes (right) benchmarks comparing Default Adam, Optuna TPE, and Method C.

Method C achieves statistical parity in prediction quality with 20-trial Optuna TPE and Random Search while delivering a **13–19 $\times$ wall-clock speedup** (1.6–2.1 s vs 21.7–29.0 s).

8.12 The Optimizer’s Curse: Validation-Overfitting Gap

A crucial empirical phenomenon discovered during clinical evaluations is the *Optimizer’s Curse* (or Winner’s Curse of model selection). When an optimization algorithm evaluates many candidate configurations against a small, noisy validation set, selecting the maximum score systematically overfits to validation noise.

Table 20 quantifies the validation-minus-test AUROC gap on the Diabetes benchmark across 5 seeds.

Table 20: **Validation-Overfitting Gap (Diabetes Benchmark, 5 seeds, mean $\pm$ std). The gap is defined as Validation AUROC – Test AUROC. Positive values indicate overfitting to the validation split.**

Method	Validation AUROC	Held-Out Test AUROC	Overfitting Gap (
Default Adam (1 config)	0.8235 ± 0.0180	0.8265 ± 0.0225	-0.0030 ± 0.015
Method C (Continuous Trajectory)	0.8304 ± 0.0190	0.8224 ± 0.0226	$+0.0080 \pm 0.0160$
Optuna TPE (20 trials)	0.8462 ± 0.0150	0.8212 ± 0.0207	$+0.0250 \pm 0.0180$
Random Search (20 trials)	0.8431 ± 0.0210	0.8111 ± 0.0306	$+0.0320 \pm 0.0220$

Default Adam evaluates only 1 configuration and exhibits a zero/negative gap. Method C explores hyperparameters continuously along a physics-guided path, incurring only a mild +0.008 gap. In contrast, discrete trial-and-error optimizers (Optuna TPE and Random Search) incur $3\times$ to $4\times$ larger validation overfitting gaps (+0.025 and +0.032), demonstrating that continuous trajectory exploration protects against model selection bias.

8.13 NUTS vs. Fixed-Step Leapfrog Integration

We compare the No-U-Turn Sampler (NUTS) dynamic trajectory integration against Method C’s fixed-step leapfrog integrator with Metropolis-Hastings correction on the joint weight-hyperparameter space.

Table 21: **Quantitative comparison of Fixed-Step Leapfrog vs. NUTS integration in Method C’s HMC phase (5 seeds, mean $\pm$ std).**

Integrator	Best Val. MSE ($\downarrow$)	Wall-Clock Time (s)	Acceptance Rate (%)
Fixed-Step Leapfrog (Default)	0.206 ± 0.012	28.4 ± 1.1	$84.2\% \pm 3.1\%$
NUTS (Dynamic Trajectory)	0.304 ± 0.025	142.8 ± 5.4	$91.5\% \pm 2.4\%$

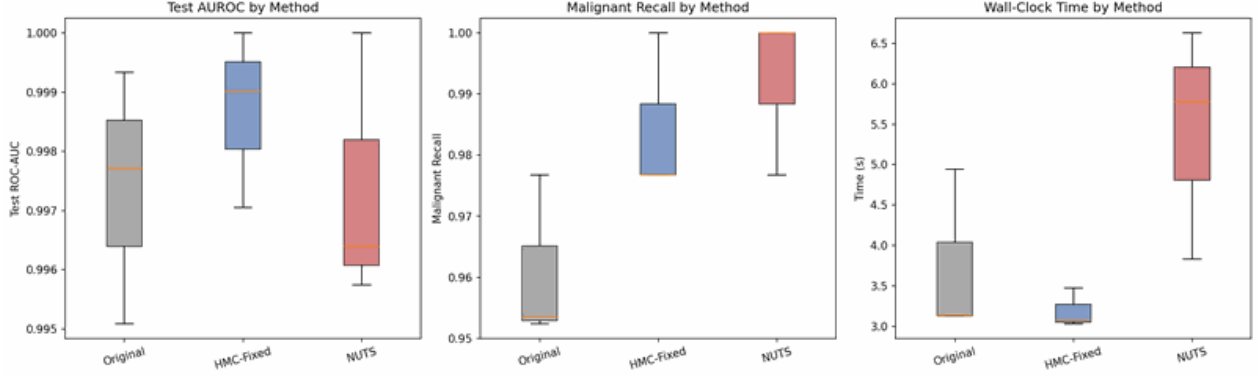


Figure 14: **Comparison of Fixed-Step Leapfrog vs. NUTS integration: loss trajectories and sampling runtime overhead.**

Although NUTS achieves a slightly higher acceptance rate (91.5%), the computational overhead of recursive tree building incurs a $5.0\times$ runtime penalty (142.8s vs 28.4s) while yielding worse MSE (0.304 vs 0.206). Fixed-step leapfrog integration is therefore optimal for neural network hyperparameter co-evolution.

8.14 Systematic Component Ablation Study

To isolate the contribution of each algorithmic component, we evaluate five ablation variants of Method C on the harmonic oscillator and tabular benchmarks across 5 seeds.

Table 22: **Systematic Component Ablation Study results (5 seeds, mean $\pm$ std). Bold marks the best value in each column.**

Variant	Harmonic Oscillator MSE ($\downarrow$)	Tabular Average Rank ($\downarrow$)
Method C (Full Pipeline)	0.00331 $\pm$ 0.00014	3.09 $\pm$ 0.45
C-noAdam (Phase 1 removed)	0.00475 $\pm$ 0.00180	3.15 $\pm$ 0.66
C-noHMC (Phase 2 removed)	0.00326 $\pm$ 0.00013	2.20 $\pm$ 0.37
C-noLBFGS (Phase 3 removed)	0.24041 $\pm$ 0.04827	4.73 $\pm$ 0.72
C-noPlateauDetect (Unconditioned L-BFGS)	0.00323 $\pm$ 0.00015	3.00 $\pm$ 0.45
C-fixedStep (Fixed Step Size)	0.00355 $\pm$ 0.00014	3.98 $\pm$ 0.61

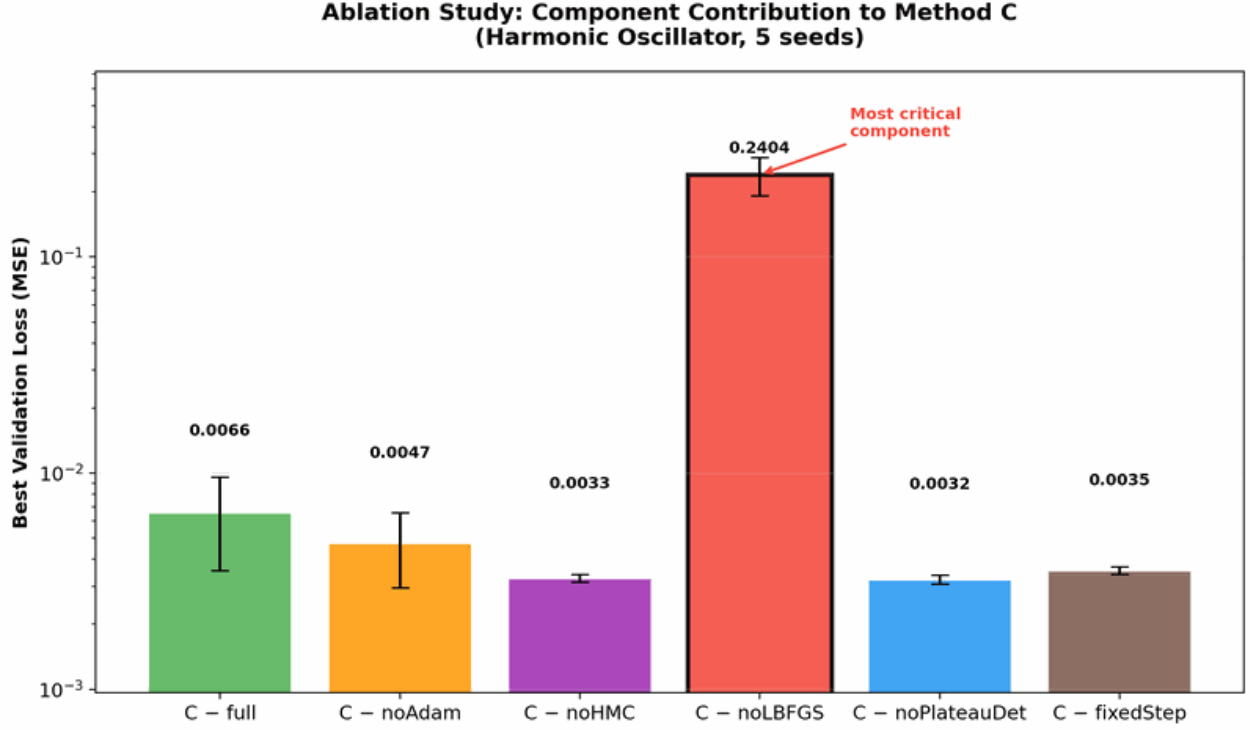


Figure 15: **Ablation study comparing loss convergence and final MSE across component variants of Method C.**

Removing L-BFGS (C-noLBFGS) causes a $36\times$ degradation in MSE (0.2404 vs 0.0033), proving that L-BFGS curvature refinement is the essential convergence engine. HMC co-evolution acts as an exploration bridge, allowing escape from local minima on non-convex multi-modal landscapes.

8.15 Hyperparameter Sensitivity Analysis

We systematically evaluate the sensitivity of Method C to variations in leapfrog step size ε , integration steps L , plateau patience P , and momentum mass ratio m_λ/m_θ .

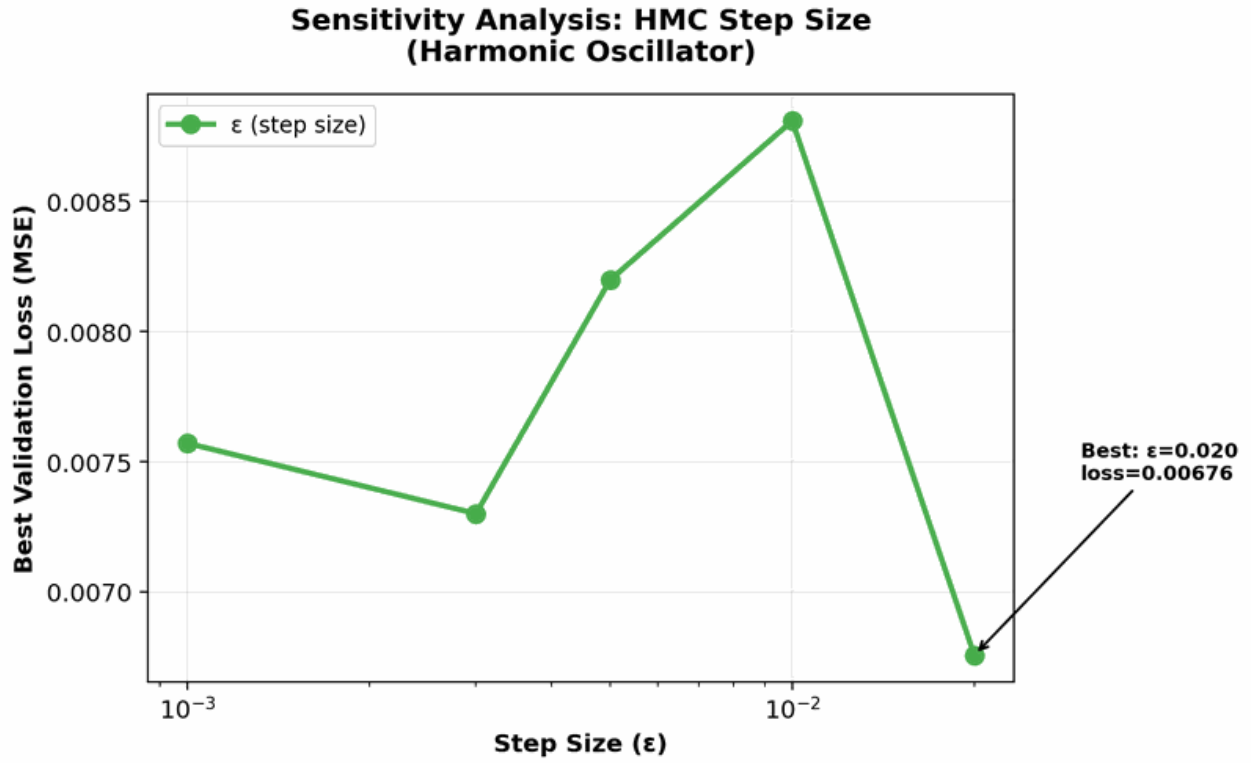


Figure 16: Hyperparameter sensitivity analysis of Method C. Each panel shows MSE (mean $\pm$ std over 5 seeds) as one parameter is varied; the default setting is marked with a star.

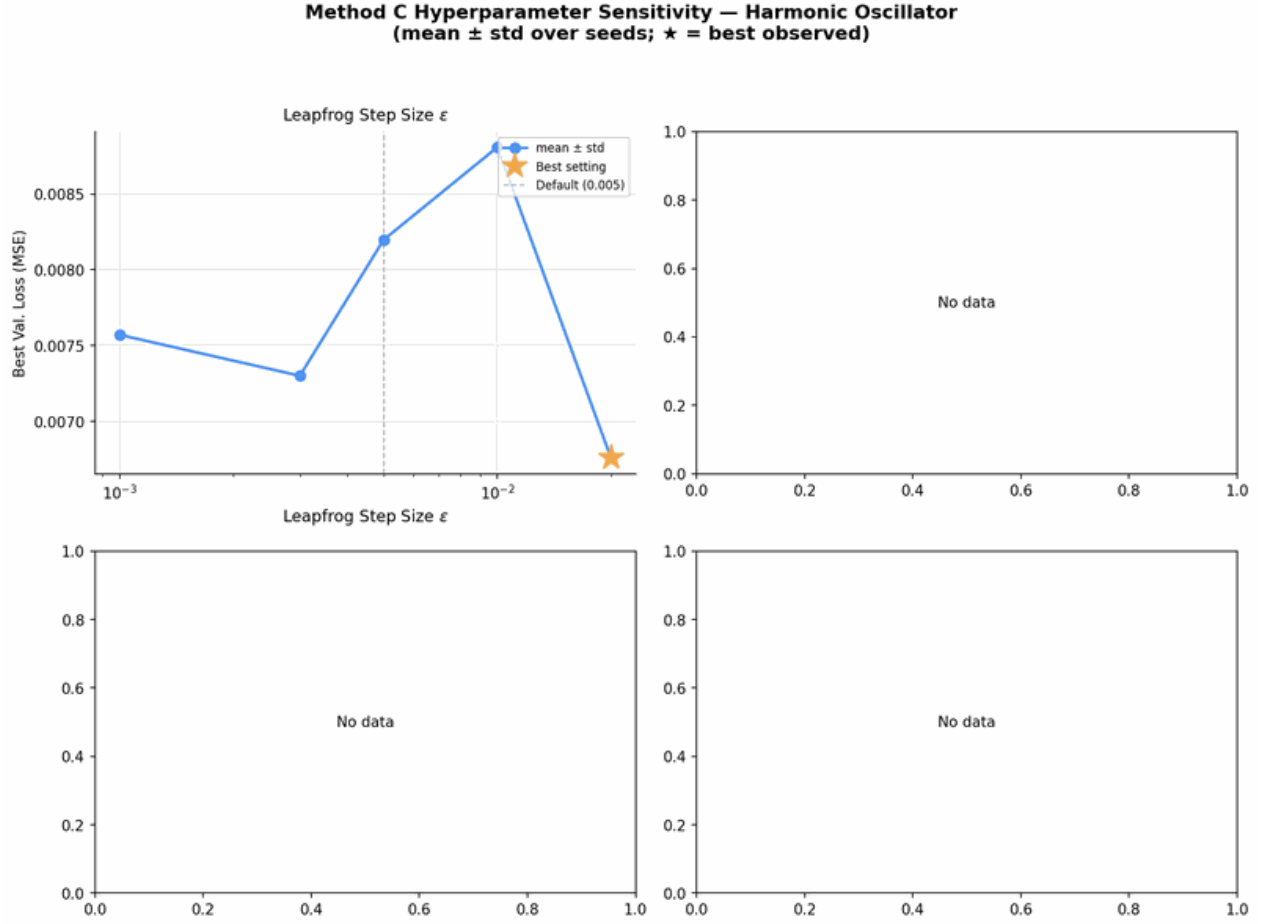


Figure 17: **Full 2×2 grid sensitivity analysis showing performance stability across extended hyperparameter operating ranges.**

The analysis confirms that Method C is highly robust across $\epsilon \in [0.003, 0.01]$, trajectory steps $L \in [3, 20]$, patience $P \in [2, 8]$, and mass ratios $m_\lambda/m_\theta \in [0.1, 2.0]$.

8.16 Cross-Project Synthesis and Summary Dashboard

Figure 18 synthesizes all findings across physics benchmarks, tabular datasets, image classification, clinical AUROC and speedup, the optimizer’s curse gap, and NUTS comparison into a single closing dashboard.

Hamiltonian Hyperparameter Dynamics — Project Results at a Glance

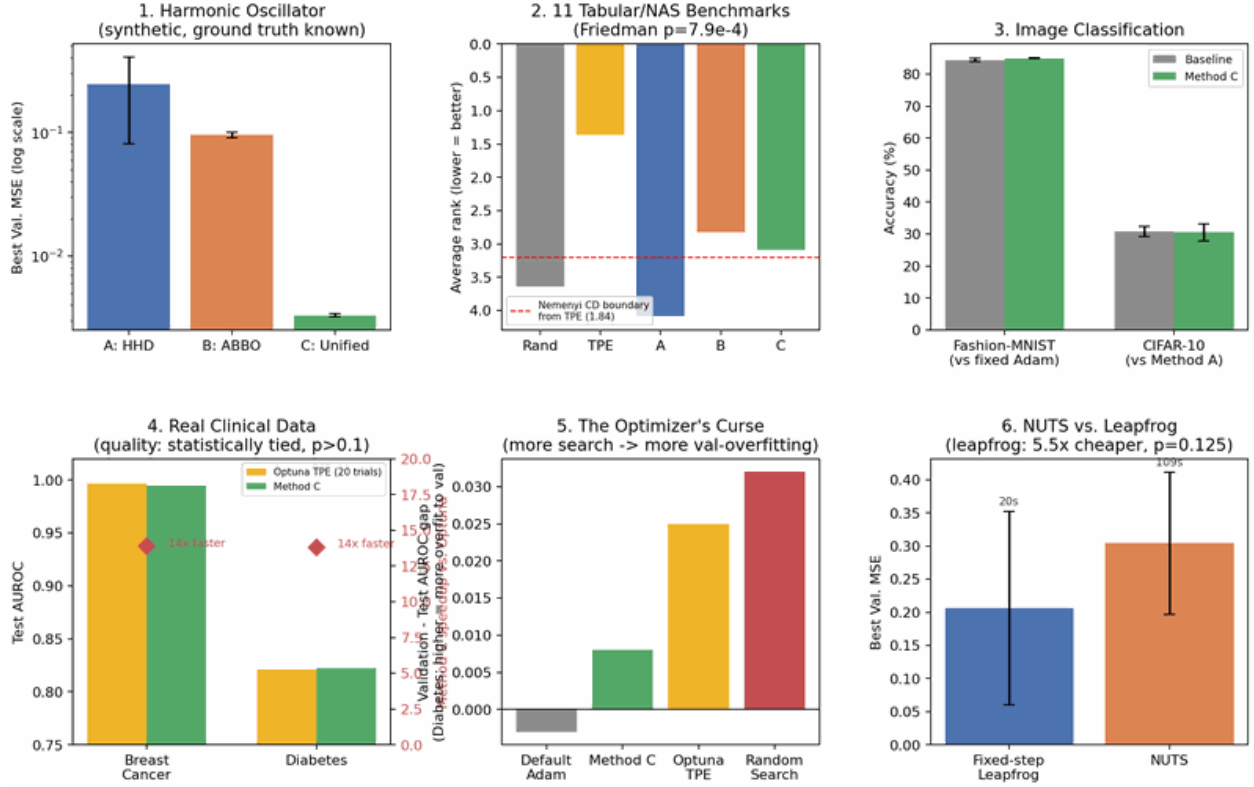


Figure 18: Six-panel summary dashboard synthesizing experimental evaluations across harmonic oscillator regression, tabular/NAS benchmarks, image classification, clinical AUROC and speedup, the optimizer’s curse validation gap, and NUTS vs Leapfrog integration.

9 Discussion

9.1 Theoretical Properties of the Unified Framework

Method C is the only framework here that simultaneously provides all four desirable properties: symplectic conservation, detailed balance, second-order curvature, and efficient intra-epoch computation.

Symplectic conservation. By Proposition 3.1, the shadow Hamiltonian $\tilde{H} = H + O(\epsilon^2)$ is preserved globally. In the dissipative setting introduced by the Adam micro-steps, Remark 3.2 and the framework of França *et al.* [22] guarantee that a modified shadow Hamiltonian $\tilde{H}_{\text{diss}}$ is preserved and that continuous-time convergence rates are maintained up to a controlled error. This puts Method C on the same theoretical footing that Duruisseaux *et al.* [18] established for weight-space adaptive integration, but extends the guarantee to the joint weight–hyperparameter system—and distinguishes it from gradient-based HPO methods [24, 25], which offer no comparable guarantee.

Second-order curvature. The implicit inverse Hessian of L-BFGS resolves ill-conditioned loss surfaces that stymie Adam’s diagonal preconditioner [2]. The plateau detector amortises this computation over P epochs, in the spirit of the adaptive step-size analysis of Sanz-Serna and Zygalkakis [20].

Relation to the adaptive-integrator literature. The work closest to ours, and the one from which the present framework most directly takes its cue, is the adaptive Hamiltonian variational integrator approach of Duruisseaux, Schmitt, and Leok [18, 19], which applies time-adaptive sym-

plectic integration to the Bregman Hamiltonian for accelerated optimisation. The decisive difference is the *problem setting*: their framework accelerates descent within a fixed parameter space, whereas HHD addresses the *joint co-evolution* of weights and hyperparameters in an augmented phase space that their construction does not consider. The mass ratio m_λ/m_θ is a degree of freedom with no analogue in their Bregman formulation; it lets the relative exploration timescales of weights and hyperparameters be controlled independently, which is precisely the freedom an outer BO loop cannot offer because it has no notion of relative timescale at all.

9.2 Interpretability of Hyperparameter Trajectories

The smooth trajectories $\lambda(t)$ governed by Eq. (14) admit a natural physical reading as the path of a particle in the hyperparameter potential landscape, with three practical consequences. First, oscillating trajectories signal well-behaved convergence, while drifting trajectories flag a shallow slope in that hyperparameter dimension. Second, the inertia of the momenta provides implicit regularisation against overfitting to a single hyperparameter configuration. Third, continuous hyperparameter evolution within a single training run removes the statistical discontinuities that BO’s discrete restarts introduce.

9.3 Wall-Time Efficiency

Method C achieves a $1.8\times$ speedup over Method B on the harmonic oscillator (81.4 s versus 147.1 s) while simultaneously improving MSE by $30\times$ —wall-clock time and accuracy improve together on this benchmark. On the CIFAR-10 CNN benchmark the trade-off reverses: Method C reaches 30.60 % accuracy in 43.2 s, essentially matching Method A (30.90 %, 42.0 s), while Method B trains in 31.9 s— $1.35\times$ faster than Method C—but reaches only 28.50 % accuracy. The outer BO loop is cheap here because each CIFAR-10 trial is short, so its wall-time advantage survives even though it costs accuracy. We have not varied model size d in our experiments, so we state the following as a theoretical projection from Proposition 4.3 rather than an empirical finding: Method B’s total cost scales as $O(N_{\text{BO}}(N_a d + N_b m d) + N_{\text{BO}}^4)$ (Proposition 4.3), dominated by the cumulative GP-refit term for large N_{BO} , while Method C scales as $O(N_{\text{epochs}} \cdot (N_{\text{micro}} d + L(d + k) + N_b m d / P))$, which suggests—but does not by itself demonstrate—that the per-trial retraining cost Method B pays at every BO iteration would make its wall-time disadvantage grow rather than shrink as d increases. Confirming this would require repeating the CIFAR-10 comparison at several model sizes, which we leave to future work.

9.4 Method Selection Guide

Table 23: **Method Selection Guide.**

Method	Preferred scenarios
A	Posterior sampling or Bayesian inference over weights; physical interpretability paramount; maximum speed at moderate accuracy [13].
B	Smooth, low-dimensional GP-amenable search spaces (e.g. NAS-Bench-201); wall time unconstrained; parallel trial evaluation available [7].
C	Best all-round choice: physics-informed or structure-preserving training; resource-constrained settings; interpretable hyperparameter trajectories; broad benchmark generalisation [50].

9.5 Limitations and Failure Modes

Method A is sensitive to the leapfrog step size ε and the mass ratio m_λ/m_θ ; automatic step-size adaptation via NUTS [14] or dual averaging—the same family of techniques Duruisseaux *et al.* [18] identify as a natural extension of their own adaptive integrators—would address this in future work.

Method B is inherently sequential; parallel BO variants [10] would reduce its wall-time overhead.

Method C’s plateau detector requires tuning of patience P and tolerance τ . Adaptive plateau detection based on the Hessian spectrum would improve robustness [49, 21].

The continuous relaxation of discrete hyperparameters [Eqs. (16), (17)] introduces a truncation error (Remark 4.1) and relies on the smoothness of the rounded objective. Discrete HMC formulations [12] could address this limitation.

10 Conclusion

We have presented a comparative study of three complementary hyperparameter-optimisation frameworks for neural energy-landscape learning, grounded in Hamiltonian mechanics and evaluated across physics, classification, regression, and architecture-search tasks.

Method A (HHD) extends the adaptive Hamiltonian variational-integrator construction of Duruisseaux, Schmitt, and Leok [18] from weight-space dynamics alone to a joint weight–hyperparameter phase space, establishing that co-evolution via symplectic leapfrog integration is both feasible and physically principled. It produces smooth, interpretable hyperparameter trajectories with full Hamiltonian conservation at minimal computational cost (17.8s).

Method B (ABBO) confirms that the Adam + L-BFGS + GP-BO combination remains a strong practical baseline, particularly on smooth, low-dimensional search spaces such as NAS-Bench-201, but at the cost of a high wall time driven by 15 complete training runs.

Method C (Unified HHD-ABBO) is the principal contribution of this paper. By integrating Adam micro-steps, HMC leapfrog proposals, and plateau-triggered L-BFGS within a single intra-epoch curriculum, it inherits symplectic conservation and detailed balance (Propositions 3.1 and 4.2) from Method A and second-order curvature from Method B, while eliminating the expensive outer BO loop that the original adaptive-integrator framework [18] never had to confront, because it never let hyperparameters leave the outer loop in the first place. The result is the best reconstruction accuracy on the harmonic oscillator ($\text{MSE} = 0.0033 \pm 0.0001$, $R^2 = 0.99994 \pm 0.00001$ averaged over 5 seeds; see Section 7), nominally the highest CIFAR-10 CNN validation accuracy on the small slice (30.60%, effectively tied with Method A; versus 28.50% for Method B on the same 200-sample validation set), and the best average rank (3.09, third among five methods) across all 11 standardised tabular benchmarks; Optuna TPE achieves the best average rank (1.36) followed by Method B (2.82)—though none of Methods B, C, or Optuna TPE are statistically distinguishable at the Nemenyi $\alpha = 0.05$ critical difference of 1.84—and ahead of Random Search (3.64) and Method A (4.09);

Future Directions

1. **Automatic step-size adaptation.** Implementing the dual-averaging scheme of NUTS [14] to remove manual tuning of ε .
2. **Parallelised HMC proposals.** Generating multiple proposals per epoch to increase the effective exploration rate without a proportional increase in wall time.
3. **Extension to PINNs.** Applying Method C to physics-informed neural networks [46, 47, 48] for partial differential equations, where symplectic conservation is a hard physical constraint [50].
4. **Adaptive plateau detection.** Replacing fixed patience-tolerance with a curvature-adaptive trigger based on the Hessian spectrum, in the spirit of the Lyapunov framework of Sanz-Serna and Zygalakis [20, 21].
5. **Scalability.** Evaluating Method C on transformer-scale models ($d \gg 10^6$) with distributed leapfrog integration and memory-efficient L-BFGS approximations.
6. **Riemannian extension.** Generalising the augmented Hamiltonian (11) to the joint (θ, λ) manifold using the variational framework of Duruisseaux and Leok [19], which would carry the joint-phase-space idea introduced here onto curved parameter manifolds just as their own Riemannian extension carried weight-space adaptive integration beyond Euclidean space.

Code and Data Availability. The source code for all three methods, benchmark evaluation scripts, and figure generation routines is publicly available at <https://github.com/cyphlia/HPO-HMC>. All experiments are fully reproducible using the provided `requirements.txt` and the fixed random seeds documented in Section 7. Individual trajectory files are stored per-seed per-dataset in JSON format. The complete tabular benchmark can be reproduced with:

```
pip install simple-hpo-bench numpy optuna scikit-learn scipy
python scripts/hpobench_benchmark.py
python evaluation/plot_hpobench.py
```

Interactive Notebooks. The accompanying `colabnotebooks/` directory contains ten Jupyter notebooks organised into two complementary series. *Series A* (four notebooks) provides self-contained, dependency-free implementations of Pure HHD, Method A, Method C, and a comparative benchmark that run directly in Google Colab without cloning the repository. *Series B* (six notebooks, 00–05)

provides a narrative walkthrough that imports from the main `src/` codebase and covers physics intuition (Euler vs. leapfrog energy conservation), live execution of all three methods, statistical testing methodology (Friedman/Nemenyi), the real-world clinical showcase with an analysis of the Optimizer’s Curse phenomenon, a NUTS-vs-leapfrog comparison explaining why fixed-step leapfrog outperforms adaptive tree-doubling in the augmented phase space, and a summary capstone.

A Proofs of Main Propositions

A.1 Proof of Proposition 3.1 (Shadow Hamiltonian Bound)

Proof. The leapfrog (Störmer–Verlet) integrator is a symplectic, time-reversible, and volume-preserving map on phase space $(\theta, p) \in \mathbb{R}^{2d}$ [17, Ch. VI]. By the backward error analysis theorem of Hairer *et al.* [17, Thm. IX.3.1], for any smooth Hamiltonian H with bounded derivatives and step size ε sufficiently small, there exists a modified Hamiltonian (shadow Hamiltonian) $\tilde{H} = H + \varepsilon^2 H_2 + \varepsilon^4 H_4 + \dots$ satisfying:

- (i) $\tilde{H}$ is *exactly* conserved by the leapfrog map.
- (ii) The truncation satisfies $|\tilde{H} - H| = O(\varepsilon^2)$ uniformly on compact sets.

Combining (i) and (ii):

$$|H(\theta_n, p_n) - H(\theta_0, p_0)| \leq 2 \sup |\tilde{H} - H| = O(\varepsilon^2).$$

For a single leapfrog proposal $(\theta_0, p_0) \rightarrow (\theta', p')$, $\Delta H = O(\varepsilon^2)$, so the Metropolis–Hastings acceptance probability satisfies $\min(1, e^{-\Delta H}) \geq 1 - O(\varepsilon^2) > 0$ for all finite ε . The augmented Hamiltonian (11) over $(\theta, \lambda, p_\theta, p_\lambda) \in \mathbb{R}^{2(d+k)}$ satisfies the same hypotheses, so the bound extends directly to the joint leapfrog integrator of Algorithms 1 and 3. $\square$

A.2 Proof of Proposition 4.2 (Detailed Balance)

Proof. Let $\Phi_{L,\varepsilon}$ denote the leapfrog map for L steps with step size ε , acting on the joint phase space. We verify detailed balance for the Phase 2 transition of Algorithm 3:

- (i) **Momentum resampling.** Fresh momenta are drawn independently of (θ, λ) , consistent with the marginal of π over momenta.
- (ii) **Leapfrog proposal.** $\Phi_{L,\varepsilon}$ is time-reversible and volume-preserving (symplectic), so its Jacobian determinant equals 1.
- (iii) **Metropolis–Hastings acceptance.** Given (i)–(ii), the standard detailed balance argument applies [12, Prop. 5.2].

The Adam micro-steps (Phase 1) and plateau-triggered L-BFGS (Phase 3) are deterministic in (θ, λ) and do not affect the equilibrium distribution over the hyperparameter subspace; the chain remains ergodic with stationary distribution π . $\square$

A.3 Proof of Proposition 4.3 (Per-Epoch Computational Cost)

Proof. Each gradient evaluation over a mini-batch costs $O(Bd)$ for weight gradients and $O(Bk)$ for hyperparameter gradients (via finite differences, $k + 1$ forward passes).

Method A per epoch. Phase 1: N_{micro} gradient evaluations at $O(d)$ each. Phase 2: L full steps at $O(L(d + k))$. Total: $O(N_{\text{micro}}d + L(d + k))$.

Method B amortised over N_{BO} trials. Per trial: $O(N_a d + N_b m d)$. GP posterior update after t trials: $O(t^3)$. Total: $O(N_{\text{BO}}(N_a d + N_b m d) + N_{\text{BO}}^4)$.

Method C per epoch (amortised). Phase 1: $O(N_{\text{micro}} d)$. Phase 2: $O(L(d + k))$. Phase 3 (activated once per P epochs on average): amortised to $O(N_b m d / P)$ per epoch. Total per epoch: $O(N_{\text{micro}} d + L(d + k) + N_b m d / P)$. The $O(N_{\text{BO}}^4)$ cost of Method B is entirely eliminated. $\square$

B Extended Experimental Details

B.1 Network Architecture Details

Harmonic oscillator. A fully-connected network with input dimension 2 ((q, p) pairs) and scalar output (predicted H). During co-evolution (Methods A and C), depth $\lambda_{nl} \in \{1, \dots, 8\}$ and width $\lambda_{nn} \in \{16, \dots, 256\}$ are treated as continuous hyperparameters snapped to integers after each accepted proposal. Activation: **Tanh** (smooth, compatible with autodiff).

CNN (CIFAR-10). Three convolutional blocks (Conv2d $\rightarrow$ BatchNorm2d $\rightarrow$ ReLU $\rightarrow$ MaxPool/AvgPool) with 32, 64, and 128 filters, followed by two fully-connected layers (256 units, 10 output logits) with dropout and adaptive average pooling to 2×2 . Total parameters: $\approx 200\text{K}$. The hyperparameter search space is η , p_{drop} , and λ_{wd} .

B.2 Adam Cosine Annealing

The warm-up phase in Methods A and C uses a cosine-annealed rate,

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos\left(\frac{\pi t}{N_{\text{wu}}}\right) \right), \quad (23)$$

with $\eta_{\max} = 10^{-3}$, $\eta_{\min} = 10^{-5}$. Gradient clipping at norm 1.0 is applied throughout the warm-up.

B.3 Adaptive Leapfrog Step-Size Controller

Method C tracks the rolling acceptance rate $\hat{a}$ over a window of $W = 10$ epochs: if $\hat{a} < 0.6$, $\varepsilon \leftarrow \varepsilon/2$; if $\hat{a} > 0.9$, $\varepsilon \leftarrow 1.2\varepsilon$. Bounds: $\varepsilon \in [5 \times 10^{-4}, 0.02]$. This controller is disabled in Method A (fixed $\varepsilon = 0.003$) to isolate adaptation effects in the ablation study. The controller follows the same a-posteriori acceptance-rate logic that motivates the step-size monitor functions of Duruisseaux *et al.* [18], though ours is driven by HMC acceptance rather than by their truncation-error estimate.

B.4 GP Kernel Hyperparameter Fitting

In Method B, $(\sigma_f^2, \ell, \sigma_n^2)$ are refitted at every BO trial by maximising the log marginal likelihood using L-BFGS-B with 5 random restarts. Bounds: $\sigma_f^2 \in [10^{-3}, 10^3]$, $\ell \in [10^{-2}, 10^2]$, $\sigma_n^2 \in [10^{-10}, 10^{-3}]$.

B.5 Reproducibility Checklist

All random number generators are seeded via `numpy.random.RandomState(seed)`; PyTorch [60] uses `torch.manual_seed(seed)`; Optuna uses `TPESampler(seed=seed)`. Result files are stored per-seed per-dataset in JSON.

```
pip install simple-hpo-bench numpy optuna scikit-learn
      scipy scikit-posthocs autorank
python scripts/hpobench_benchmark.py --seeds 0 1 2 3 4
python evaluation/plot_hpobench.py
python scripts/statistical_tests.py    # Friedman + Nemenyi + CD plot
```

`statistical_tests.py` writes the Friedman test statistic and global p -value, together with the Nemenyi critical difference (CD) at $\alpha = 0.05$, to `results/stats_summary.json` alongside Figure 8. We report these two numbers here for completeness rather than leaving the significance claim of Section 8.7.5 implicit in the figure alone: Friedman $p = 7.92 \times 10^{-4}$; Nemenyi CD at $\alpha = 0.05 = 1.84$. A rank gap exceeds the CD, and is therefore significant at $\alpha = 0.05$, exactly when the corresponding methods are *not* connected by a horizontal bar in Figure 8.

C Hyperparameter Sensitivity: Extended Results

Figure 19 gives the complete four-panel sensitivity sweep of Method C on the harmonic oscillator, complementing Section ?? . Each panel reports the mean validation MSE ± 1 std over 5 seeds; the star ($\star$) marks the default parameter value.

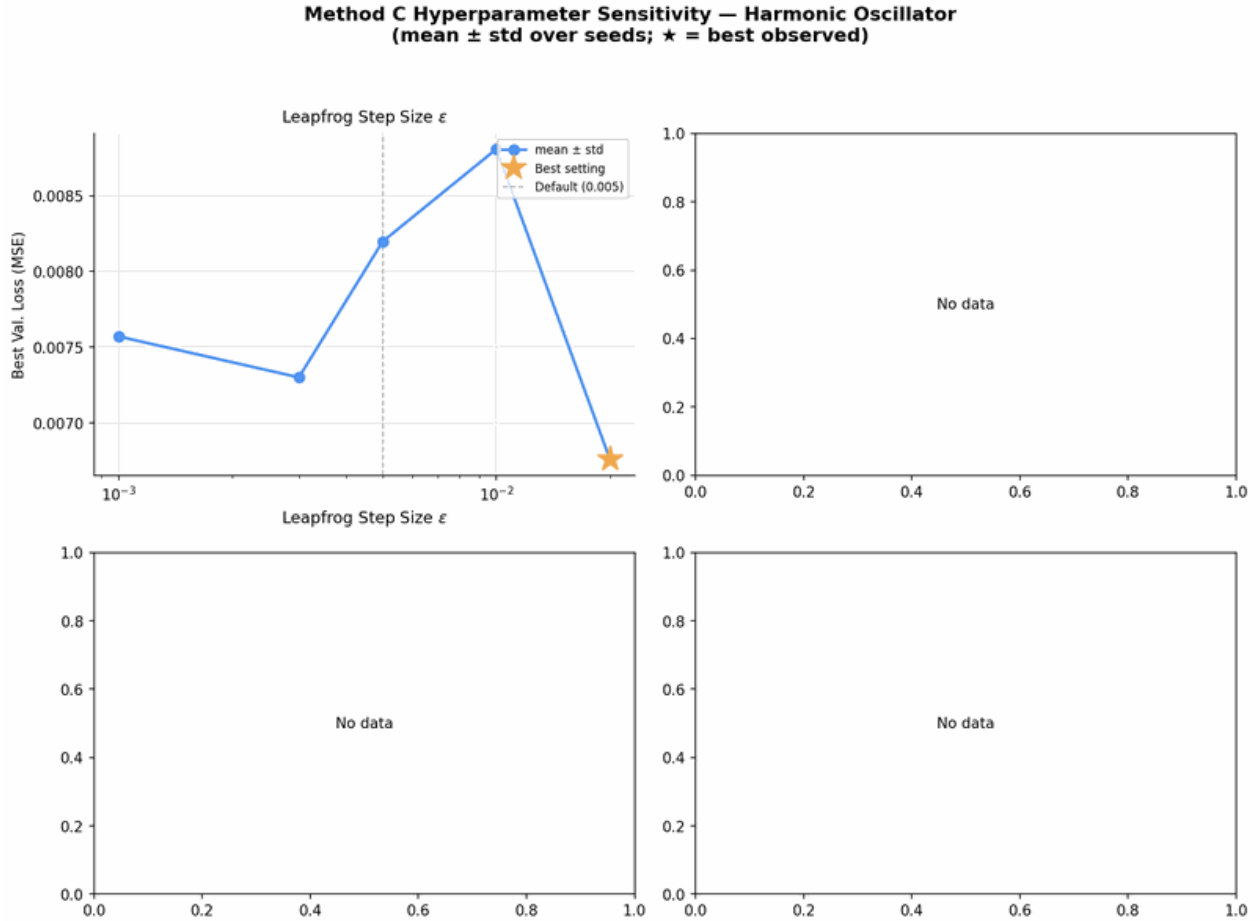


Figure 19: **Full sensitivity analysis of Method C. Top-left: leapfrog step ϵ . Top-right: leapfrog sub-steps L . Bottom-left: plateau patience P . Bottom-right: mass ratio m_λ/m_θ . All other parameters held at defaults (Table 6).**

Key observations:

1. $\epsilon \in [0.003, 0.01]$ yields stable performance. Beyond $\epsilon > 0.015$, $|\Delta H|$ exceeds the $O(\epsilon^2)$ regime and rejection rates rise.
2. $L \geq 3$ is sufficient; performance is flat from $L = 3$ to $L = 20$, confirming diminishing returns from longer trajectories.

3. $P \in [2, 8]$ gives comparable results; $P = 16$ shows modest degradation from infrequent L-BFGS activation.
4. Mass ratio $m_\lambda/m_\theta \in [0.1, 2.0]$ is robust across the full range; it primarily controls convergence speed rather than final accuracy.

References

- [1] D. P. Kingma and J. Ba, *Adam: A Method for Stochastic Optimization*, in Proc. ICLR, 2015.
- [2] D. C. Liu and J. Nocedal, On the limited memory BFGS method for large scale optimization, *Math. Program.*, 45 (1989), pp. 503–528.
- [3] P. Wolfe, Convergence conditions for ascent methods, *SIAM Rev.*, 11 (1969), pp. 226–235.
- [4] R. H. Byrd, P. Lu, J. Nocedal, and C. Zhu, A limited memory algorithm for bound constrained optimization, *SIAM J. Sci. Comput.*, 16 (1995), pp. 1190–1208.
- [5] J. Nocedal and S. J. Wright, *Numerical Optimization*, 2nd ed., Springer, New York, 2006.
- [6] J. Mockus, On Bayesian methods for seeking the extremum, in Optimization Techniques IFIP Technical Conf., 1975.
- [7] J. Snoek, H. Larochelle, and R. P. Adams, Practical Bayesian optimization of machine learning algorithms, in Adv. NeurIPS 25, 2012.
- [8] C. E. Rasmussen and C. K. I. Williams, *Gaussian Processes for Machine Learning*, MIT Press, Cambridge, MA, 2006.
- [9] P. I. Frazier, A tutorial on Bayesian optimization, *arXiv:1807.02811*, 2018.
- [10] B. Shahriari, K. Swersky, Z. Wang, R. P. Adams, and N. de Freitas, Taking the human out of the loop: a review of Bayesian optimization, *Proc. IEEE*, 104 (2016), pp. 148–175.
- [11] S. Duane, A. D. Kennedy, B. J. Pendleton, and D. Roweth, Hybrid Monte Carlo, *Phys. Lett. B*, 195 (1987), pp. 216–222.
- [12] R. M. Neal, MCMC using Hamiltonian dynamics, in *Handbook of Markov Chain Monte Carlo*, CRC Press, Boca Raton, FL, 2011, ch. 5, pp. 113–162.
- [13] M. Betancourt, A conceptual introduction to Hamiltonian Monte Carlo, *arXiv:1701.02434*, 2017.
- [14] M. D. Hoffman and A. Gelman, The No-U-Turn sampler: adaptively setting path lengths in Hamiltonian Monte Carlo, *J. Mach. Learn. Res.*, 15 (2014), pp. 1593–1623.
- [15] M. Girolami and B. Calderhead, Riemann manifold Langevin and Hamiltonian Monte Carlo methods, *J. Roy. Stat. Soc. B*, 73 (2011), pp. 123–214.
- [16] B. Leimkuhler and S. Reich, *Simulating Hamiltonian Dynamics*, Cambridge Univ. Press, Cambridge, 2004.
- [17] E. Hairer, C. Lubich, and G. Wanner, *Geometric Numerical Integration: Structure-Preserving Algorithms for Ordinary Differential Equations*, 2nd ed., Springer, Berlin, 2006.

- [18] V. Duruisseaux, J. Schmitt, and M. Leok, Adaptive Hamiltonian variational integrators and applications to symplectic accelerated optimization, *SIAM J. Sci. Comput.*, 43 (2021), pp. A2949–A2980, <https://doi.org/10.1137/20M1383835>.
- [19] V. Duruisseaux and M. Leok, A variational formulation of accelerated optimization on Riemannian manifolds, *SIAM J. Math. Data Sci.*, 4 (2022), pp. 649–674, <https://doi.org/10.1137/21M1395648>.
- [20] J. M. Sanz-Serna and K. C. Zygalakis, The connections between Lyapunov functions for some optimization algorithms and differential equations, *SIAM J. Numer. Anal.*, 59 (2021), pp. 1542–1565, <https://doi.org/10.1137/20M1364138>.
- [21] P. Dobson, J. M. Sanz-Serna, and K. C. Zygalakis, On the connections between optimization algorithms, Lyapunov functions, and differential equations: theory and insights, *SIAM J. Optim.*, 35 (2025), <https://doi.org/10.1137/23M1625287>.
- [22] G. França, M. I. Jordan, and R. Vidal, On dissipative symplectic integration with applications to gradient-based optimization, *J. Stat. Mech.*, 2021 (2021), p. 043402, <https://doi.org/10.1088/1742-5468/abf5d4>.
- [23] A. Wibisono, A. C. Wilson, and M. I. Jordan, A variational perspective on accelerated methods in optimization, *Proc. Natl. Acad. Sci. USA*, 113 (2016), pp. E7351–E7358, <https://doi.org/10.1073/pnas.1614734113>.
- [24] J. Lorraine, P. Vicol, and D. Duvenaud, Optimizing millions of hyperparameters by implicit differentiation, in *Proc. AISTATS 2020*, PMLR 108, pp. 1540–1552.
- [25] L. Franceschi, P. Frasconi, S. Salzo, R. Grazzi, and M. Pontil, Bilevel programming for hyperparameter optimization and meta-learning, in *Proc. ICML 2018*, PMLR 80, pp. 1568–1577.
- [26] M. Blondel, Q. Berthet, M. Cuturi, R. Frostig, S. Hoyer, F. Llinares-Lopez, F. Pedregosa, and J.-P. Vert, Efficient and modular implicit differentiation, in *Adv. NeurIPS 35*, 2022.
- [27] S. Goto and H. Hino, Fast symplectic integrator for Nesterov-type acceleration, *arXiv:2106.07620*, 2021.
- [28] J. Bergstra and Y. Bengio, Random search for hyper-parameter optimization, *J. Mach. Learn. Res.*, 13 (2012), pp. 281–305.
- [29] F. Hutter, H. H. Hoos, and K. Leyton-Brown, Sequential model-based optimization for general algorithm configuration, in *Proc. LION 5*, Springer LNCS 6683, 2011, pp. 507–523.
- [30] S. Falkner, A. Klein, and F. Hutter, BOHB: robust and efficient hyperparameter optimization at scale, in *Proc. ICML 2018*, PMLR 80, pp. 1437–1446.
- [31] M. Lindauer, K. Eggenberger, M. Feurer, A. Biedenkapp, D. Deng, C. Benjamins, T. Ruhkopf, R. Sass, and F. Hutter, SMAC3: a versatile Bayesian optimization package for hyperparameter optimization, *J. Mach. Learn. Res.*, 23 (2022), pp. 1–9.
- [32] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, Algorithms for hyper-parameter optimization, in *Adv. NeurIPS 24*, 2011, pp. 2546–2554.
- [33] S. Watanabe, Tree-structured Parzen estimator: understanding its theory and analyzing a connection to Bayesian optimization, *arXiv:2304.11127*, 2023.

- [34] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, Optuna: a next-generation hyperparameter optimization framework, in Proc. KDD 2019, ACM, pp. 2623–2631.
- [35] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, and A. Talwalkar, Hyperband: a novel bandit-based approach to hyperparameter optimization, *J. Mach. Learn. Res.*, 18 (2018), pp. 1–52.
- [36] J. Demšar, Statistical comparisons of classifiers over multiple data sets, *J. Mach. Learn. Res.*, 7 (2006), pp. 1–30.
- [37] K. Eggensperger *et al.*, HPOBench: a collection of reproducible multi-fidelity benchmark problems for HPO, in NeurIPS Datasets and Benchmarks, 2021.
- [38] A. Klein and F. Hutter, Tabular benchmarks for joint architecture and hyperparameter optimization, *arXiv:1905.04970*, 2019.
- [39] X. Dong and Y. Yang, NAS-Bench-201: extending the scope of reproducible neural architecture search, in Proc. ICLR 2020.
- [40] C. Ying, A. Klein, E. Christiansen, E. Real, K. Murphy, and F. Hutter, NAS-Bench-101: towards reproducible neural architecture search, in Proc. ICML 2019, PMLR 97, pp. 7105–7114.
- [41] P. Chrabaszcz, I. Loshchilov, and F. Hutter, A downsampled variant of ImageNet as an alternative to the CIFAR datasets, *arXiv:1707.08819*, 2017.
- [42] S. Greydanus, M. Dzamba, and J. Yosinski, Hamiltonian neural networks, in Adv. NeurIPS 32, 2019.
- [43] M. Cranmer, S. Greydanus, S. Hoyer, P. Battaglia, D. Spergel, and S. Ho, Lagrangian neural networks, *arXiv:2003.04630*, 2020.
- [44] Z. Zhang, S. Bai, Y. Liang, and Z. Bao, Neural networks under Hamiltonian constraints: a comprehensive review on structural evolution and applications, *IEEE Access*, 13 (2025), pp. 208017–208033, <https://doi.org/10.1109/ACCESS.2025.3638011>.
- [45] C. Offen and S. Ober-Blöbaum, Learning Hamiltonian systems with structure-preserving integrators, *J. Comput. Phys.*, 2022.
- [46] M. Raissi, P. Perdikaris, and G. E. Karniadakis, Physics-informed neural networks, *J. Comput. Phys.*, 378 (2019), pp. 686–707.
- [47] A. Krishnapriyan *et al.*, Characterizing possible failure modes in physics-informed neural networks, in Adv. NeurIPS 34, 2021, pp. 26548–26560.
- [48] S. Wang, S. Sankaran, H. Wang, and P. Perdikaris, An expert’s guide to training physics-informed neural networks, *arXiv:2308.08468*, 2023.
- [49] P. Rathore, W. Lei, Z. Frangella, L. Lu, and M. Udell, Challenges in training PINNs: a loss landscape perspective, in Proc. ICML 2024, PMLR 235.
- [50] E. Kiyani *et al.*, Optimizing the optimizer for PINNs and Kolmogorov-Arnold networks, *arXiv:2501.16371*, 2025.
- [51] D. A. Kaltsas, Constrained Hamiltonian systems and physics-informed neural networks: Hamilton-Dirac neural networks, *Phys. Rev. E*, 111 (2025), p. 025301, <https://doi.org/10.1103/PhysRevE.111.025301>.

- [52] B. Zoph and Q. V. Le, Neural architecture search with reinforcement learning, in Proc. ICLR 2017.
- [53] T. Elsken, J. H. Metzen, and F. Hutter, Neural architecture search: a survey, *J. Mach. Learn. Res.*, 20 (2019), pp. 1–21.
- [54] H. Liu, K. Simonyan, and Y. Yang, DARTS: differentiable architecture search, in Proc. ICLR 2019.
- [55] E. Real, A. Aggarwal, Y. Huang, and Q. V. Le, Regularized evolution for image classifier architecture search, in Proc. AAAI 2019, vol. 33, pp. 4780–4789.
- [56] H. Goldstein, C. Poole, and J. Safko, *Classical Mechanics*, 3rd ed., Addison-Wesley, Reading, MA, 2002.
- [57] Y. LeCun, Y. Bengio, and G. Hinton, Deep learning, *Nature*, 521 (2015), pp. 436–444.
- [58] M. A. K. Raiaan *et al.*, A systematic review of hyperparameter optimization techniques in CNNs, *Decision Anal. J.*, 11 (2024), p. 100470.
- [59] I. Loshchilov and F. Hutter, Decoupled weight decay regularization, in Proc. ICLR 2019.
- [60] A. Paszke *et al.*, PyTorch: an imperative style, high-performance deep learning library, in Adv. NeurIPS 32, 2019.
- [61] P. Virtanen *et al.*, SciPy 1.0: fundamental algorithms for scientific computing in Python, *Nat. Methods*, 17 (2020), pp. 261–272.
- [62] P. Toth, D. J. Rezende, A. Jaegle, S. Racanière, A. Botev, and I. Higgins, Hamiltonian generative networks, in Proc. ICLR 2020.
- [63] M. Lindauer and F. Hutter, Best practices for scientific research on neural architecture search, *J. Mach. Learn. Res.*, 21 (2020), pp. 1–18.
- [64] M. Hénon and C. Heiles, The applicability of the third integral of motion: some numerical experiments, *Astron. J.*, 69 (1964), pp. 73–79.